scientific reports

Check for updates

OPEN

A novel quantum convolutional neural network framework for quantum-enhanced classification of pixelated colour images

Chisomo Daka[1] & Somnath Bhattacharyya[1,2]✉

As image datasets grow in scale and complexity, classical convolutional neural networks (CNNs) increasingly face limitations in computational efficiency, scalability, and generalisation in low-data regimes. Quantum machine learning (QML) offers a promising alternative, with quantum convolutional neural networks (QCNNs) exploiting quantum parallelism and entanglement for feature extraction. This paper introduces the Novel Quantum Convolutional Neural Network (No-QCNN), which differs from recent hybrid QCNN/QCQ-CNN architectures by employing an end-to-end quantum convolution-pooling pipeline without inserting an intermediate classical CNN block. No-QCNN is a hybrid quantum–classical model that employs a variational quantum classifier (VQC) optimised via the COBYLA algorithm and is designed for binary and multiclass classification of low-resolution colour images on near-term quantum devices. A key innovation is a problem-specific quantum feature map that pre-processes each image into a structured three-dimensional block-matrix representation, jointly encoding pixel colour (R, G, B) and spatial position before mapping this information into a hierarchical *ZZFeatureMap*. This encoding captures spatial-chromatic correlations at shallow circuit depth, making it compatible with noisy intermediate-scale quantum (NISQ) constraints. The model is implemented using IBM-Qiskit on a local quantum simulator and benchmarked against a classical CNN. For a six-class classification task on a 50-image dataset, No-QCNN achieves a validation accuracy of 82.05%, substantially outperforming the classical CNN's 40.00%, indicating improved generalisation and reduced overfitting in low-data, multiclass settings. Conversely, for a simpler binary classification task, the classical CNN achieves perfect validation accuracy of 100%, surpassing the No-QCNN's 89.7%. We further observe that No-QCNN performance declines with increasing dataset size and training time due to limited circuit expressibility, data-encoding overhead, and finite-sampling noise inherent to NISQ models. Overall, these results position No-QCNN as a complementary framework best suited to low-data, correlation-rich classification tasks, defining a realistic niche for quantum-enhanced artificial vision and quantum perception in the NISQ era.

Keywords Novel quantum convolutional neural network, Qubit, Image classification, Variational quantum classifier

Background

The "visible world" is nothing more than images. Images, as ubiquitous as they are, are the primary medium through which information is perceived, processed, and interpreted. Many aspects of our daily lives rely on images, with some applications ranging from medical diagnostics to autonomous systems, security surveillance, and entertainment[1–5]. Even so, images contain valuable information wherefrom meaning and patterns can be extracted. This is an area where classical machine learning techniques, particularly convolutional neural networks (CNNs), have achieved remarkable success in image classification tasks by demonstrating their capacity with complex hierarchical feature extraction and pattern recognition, as illustrated in Fig. 1[6–8]. However, as image datasets grow exponentially (in both size and complexity), classical deep learning models are reportedly faced with serious challenges in terms of computational efficiency, energy consumption, and scalability[9]. On the other

[1]Nano-Scale Transport Physics Laboratory, School of Physics, University of the Witwatersrand (Wits), Johannesburg, 1 Jan Smuts Ave, Braamfontein, Johannesburg 2050, South Africa. [2]Present address: Quantum Technology Division, SRM University-AP, Amaravati, Andhra Pradesh 522240 Mangalagiri Mandal, India. ✉email: Somnath.Bhattacharyya@wits.ac.za; bhattacharyya.s@srmap.edu.in

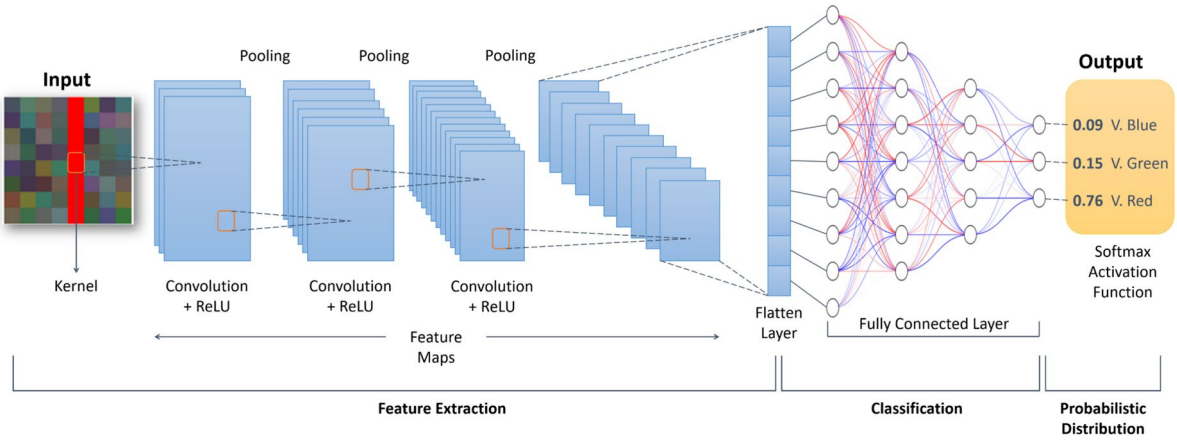


Fig. 1. A typical layout of a classical convolutional neural network (CNN) architecture for the classification of multiclass image datasets with vertical and horizontal lines of red, blue, and green colours with a noisy background. This architecture was implemented using TensorFlow to benchmark the performance metrics of No-QCNN with a comparable classical CNN in Sect. "No QCNN vs. classical CNN: a comparative analysis".

hand, the fast advancement of quantum computing technologies offers a promising alternative with which some of these limitations can be addressed. By exploiting quantum superposition, entanglement, and parallelism, quantum computing devices have the potential to considerably enhance computational efficiency. This enhanced capability could redefine the paradigm of image classification and artificial/computer vision, allowing for faster processing of much larger datasets and the discovery of more complex patterns[10–15].

Quantum machine learning (QML) has emerged as a frontier research area, combining the principles of quantum computing with machine learning models to solve high-dimensional problems more efficiently than their classical counterparts[16,17]. Among the various QML architectures, quantum convolutional neural networks (QCNNs) have gained considerable attention for their potential to mimic classical CNNs while leveraging quantum speedup. The core idea behind QCNNs is to prepare a quantum feature map by embedding classical data into its quantum state representation, and then replace classical convolutional and pooling layers with single and/or two-qubit unitary operations (e.g., Pauli X, Y, or Z rotations) on a parameterised quantum circuit (i.e., ansatz). A series of alternating convolutional and pooling layers is applied on the quantum circuit until the dimensionality of the QCNN is greatly reduced to a few qubits that are measured in the z-basis[18–20]. For near-term devices, where coherence times are short-lived due to noise, which ultimately introduces errors, the training process mainly involves optimisation of the ansatz parameters by using classical optimisers to reduce eigenvalues of a cost function; a similar technique is used in a broader category of variational quantum algorithms (VQAs)[21–23]. Despite the credibility of this quantum–classical hybrid approach, existing QCNN models are often constrained by their reliance on small-scale datasets, limited expressibility of quantum feature maps, and challenges associated with *noisy intermediate-scale quantum* (NISQ) devices[24,25].

Motivation behind No-QCNN

In an attempt to address some of these constraints, we introduce a novel quantum convolutional neural network (No-QCNN) that optimises quantum feature extraction, integrates hybrid quantum–classical learning, and implements quantum convolutional-pooling strategies to enhance multiclass image classification performance, particularly for low-resolution, pixelated $2^n \times 2^n$ colour images (where $n \in \{2, 3\}$). In this case, our underlying motivation for focusing on these low-resolution images emanates from their increasing relevance in applications such as embedded vision systems, digital iconography, low-resolution object recognition (e.g., low-resolution satellite imagery for remote sensing), and resource-constrained image processing environments[26–28]. As opposed to large-scale image datasets that require extensive computational resources, these small-scale images present an ideal testbed for evaluating the feasibility of quantum-enhanced image classification. Despite this, the limited number of pixels in 8×8 images (i.e., $n = 3$) present unique challenges to traditional CNNs due to their reliance on deep hierarchical feature representations that may not ordinarily be viable for such low-resolution inputs (unless treated with special algorithms[29]). By leveraging the supremacy of quantum phenomena, No-QCNN introduces an efficient quantum feature encoding mechanism that captures essential spatial and chromatic relationships within these images while mitigating the loss of critical information during quantum downsampling through pooling operations.

It has to be noted that the proposed No-QCNN primarily stems from the quantum feed-forward neural network (QFFNN) framework presented in our recent work on simulating the Diamond NV center Hamiltonian using a quantum neural network-inspired variational quantum circuit[30]. In this design, the multi-layer variational quantum circuit ansatz unitary operations are systematically transformed and partitioned to yield efficient two-qubit entangling operations that constitute the convolutional and pooling layers of the No-QCNN. Firstly, we prepare the classical image dataset and pre-process the images into a single row block matrix representation having 2^n rows and 2^n columns; with each block element representing a pixel position composed of the red (R),

green (G), and blue (B) colour pixel values. Then, the pre-processed image data is split into two sub-datasets, namely *training data* and *testing data,* to be used for training and testing the performance of the No-QCNN model, respectively. To create the final quantum feature map, the block matrices are encoded as quantum states where each combination of pixel colour and position values translates to a unique set of Pauli rotation operations. A comprehensive No-QCNN quantum feature map preparation process is illustrated in Fig. 10 (see Sect. "No-QCNN quantum feature map preparation"). The quantum feature map is then embedded as a quantum state representation of the pixelated colour images to initialise the No-QCNN model, followed by the application of alternating convolutional and pooling layers. A classical optimiser is also identified and assigned to optimise the iterative training task. The fully composed No-QCNN is fed with a quantum observable, which determines the fully connected qubit layer to be measured in each training iteration as defined by the output of the final quantum pooling layer. After training, testing is conducted with the unseen testing dataset. Finally, we compare the results of both training and testing accuracy/losses with respect to a classical CNN model to draw insights on model performance metrics.

Related work and contribution

This study makes several key contributions to the field of quantum-enhanced image classification by addressing critical limitations of existing QCNN architectures on near-term platforms. In contrast to recent hybrid QCNN and QCQ-CNN approaches that interleave quantum and classical convolutional blocks[40,65], the proposed No-QCNN is explicitly positioned as an end-to-end quantum convolution-pooling architecture designed to operate entirely within a NISQ-compatible quantum learning pipeline. First, we propose an innovative No-QCNN model that extends the capabilities of QCNNs and VQCs by introducing a modular quantum convolutional-pooling structure tailored for low-resolution, multichannel (RGB) image data. Unlike many existing QCNN implementations that primarily focus on grayscale or binary inputs and employ generic data encodings, the No-QCNN introduces a *problem-specific quantum feature map* that jointly encodes pixel colour (R, G, B) and spatial position. While some QCNN approaches lack robustness in extracting discriminative features from colour spaces[31–33], our approach incorporates a structured pre-encoding stage that transforms each image into a three-dimensional block-matrix representation prior to quantum state preparation, thereby preserving both spatial and chromatic relationships. These features are embedded using customised Pauli rotation operations within a hierarchical ZZFeatureMap, allowing rich spatial-chromatic correlations to be captured at shallow circuit depth while minimising gate count and circuit complexity.

Second, the No-QCNN architecture introduces a novel qubit downsampling mechanism using quantum pooling layers derived from optimal two-qubit entangling gates suggested in[34–36] to optimise the circuit width of its fully connected layer. This design explicitly targets the expressibility-trainability trade-off inherent to NISQ-era variational circuits, reducing circuit width and parameter count while preserving essential discriminative information. The convolution-pooling layer pairs are deliberately constructed to reduce dimensionality without the computational overhead and overfitting behaviour commonly associated with deep classical CNN hierarchies, thereby positioning No-QCNN as particularly effective in low-data, correlation-rich learning regimes. Third, we experimentally validate the proposed framework using IBM's Qiskit SDK, demonstrating the feasibility of No-QCNN under realistic NISQ conditions through binary and multiclass classification of pixelated colour images. Unlike reinforcement-learning-based VQC training schemes[37], we adopt a supervised learning paradigm aligned with practical image classification tasks. It is important to highlight that our hybrid training framework incorporates a gradient-free classical optimiser to iteratively update quantum circuit parameters, a choice motivated by barren-plateau mitigation and finite-coherence constraints of near-term hardware[38,39].

Finally, we benchmark No-QCNN against classical CNN baselines to provide a systematic comparison in terms of accuracy, generalisation, training stability, and parameter efficiency. The results reveal a clearly defined operational regime: while classical CNNs dominate large-scale or simple binary tasks, No-QCNN exhibits superior generalisation in low-data, multiclass scenarios where classical models are prone to overfitting. Collectively, these contributions not only demonstrate the potential of quantum computing in artificial vision and image processing but also define a realistic and complementary role for quantum-enhanced learning models, bridging the gap between quantum algorithm design and practical image classification in embedded and resource-limited environments.

Methodology: NO-QCNN architecture

This section outlines the methodical process followed to develop the No-QCNN model. In essence, the model mirrors the design principles that are found in a typical classical CNN (see Fig. 1) and integrates quantum variational techniques that are implemented through a hybrid quantum–classical framework to enable efficient image classification by detecting particular features and patterns associated with multichannel (RGB) image inputs of low resolution. The proposed approach encompasses four key stages that includes: (i) generation and pre-processing of classical image data into a block matrix representation suitable for quantum circuit encoding; (ii) data encoding and preparation of quantum feature maps using parametrized rotation gates; (iii) design of modular convolutional-pooling circuits layers; and (iv) design of the final layout of the hybrid neural network architecture including the fully connected circuit layer.

Dataset generation and preprocessing

As a first step, we needed to generate classical image data samples with known labels since our intended machine learning (i.e., training) process implements supervised learning. Two categories of pixelated image datasets were generated, one with *binary features* and the other with *multiclass features*, as inputs for binary and multiclass image classification, respectively. For the first category of image dataset, we randomly generated 4×4 pixelated images with each image having either a characteristic horizontal or vertical line pattern and a noisy background.

We assigned specific pixel values (i.e., $= \frac{\pi}{2}$) to all pixel positions representing the vertical and horizontal line patterns, with each line covering a total of 4-pixel positions. The rest of the image pixels were populated with randomly generated pixel values p in the range $[0, \frac{\pi}{4}]$ to produce a noisy background. We deliberately filled the background with random pixel values as opposed to zeros (i.e., 0 s) to avoid feeding the No-QCNN model with a perfect and simplistic image background that would make the pattern recognition task easier and trivial for training and testing processes. Pixelated images having a horizontal line feature were labelled "x" while those with a vertical line feature were labelled "y". Finally, the binary feature image dataset was successfully generated with characteristic horizontal and vertical yellow lines, where the image line orientation served as the main attribute of the binary feature class, as shown in Fig. 2 below.

The second category of our labelled image dataset was generated using a slightly different method, as compared with the first category, in order to present a lot more image features for operations with multiclass classifiers. In this case, we randomly generated 8×8 pixelated images with twice as large vertical and horizontal lines (i.e., each line feature spanned a total of 8-pixel positions in the newly generated dataset). Just like we did with the first dataset, labels "x" and "y" were also used to denote a pixelated image with a horizontal and vertical line feature, respectively. Then we went a step ahead to introduce the RGB colours for each randomly generated line feature on an image. Thus, each line feature on an image sample was assigned a randomly selected pixel value q associated with one of the three RGB colours red ($q = [255, 0, 0]$), green ($q = [0, 255, 0]$) or blue ($q = [0, 0, 255]$), with no other colour code in between, for each pixel colour q in an image's line feature. Labels "R", "G", and "B" were used to denote a pixeled image with a red, green, and blue colour, respectively. Hence, with this new image notation in place, a complete image label was defined as a *list datatype* of size 2 (i.e., the label ["x", "R"] meant a horizontal red line), giving us a total of 6 possible line-colour combinations. The rest of the image pixels were assigned a random background noise with an RGB pixel value p whose values ranged between $[0, 0, 0]$ and $[127, 127, 127]$ to avoid feeding the No-QCNN training algorithm with a perfect image background for easy pattern memory, as shown in Fig. 3. Ultimately, the generated image dataset exhibited two classes of image features. The first class of features is the *image line orientation* (which can either be vertical or horizontal), and the other feature class is the *image line colour* (which can either be red, green, or blue). This means that for every image line orientation, there's a corresponding image line colour, thus having the desired multiclass features for multiclass classifiers.

Beyond generating pixelated images with vertical and horizontal lines, we also scaled up the capabilities of the image generation algorithm by demonstrating the generation of pixelated iconised images. The pixelated images generated in this way might be used in several LCD applications like video games, websites, chat apps, etc. that leverage digital icons with low resolution. The sample of pixelated image icons shown in Fig. 4 illustrates four images, namely: *Mario*, *Battery*, *Television,* and *Cup* as read from left to right. This approach, though capable of generating images with multiple and interesting features, is perhaps beyond the scope of our present work and will thus be exploited further in future endeavours as we dive into the classification of image datasets with more diverse feature classes. However, it is worth noting the potential of this approach in generating image datasets having more complex features and patterns for robust training and testing of the No-QCNN.

Circuit feature mapping (data encoding)

Having generated and pre-processed the classical image dataset, we are now faced with the task of transforming classical image information into its quantum state representation. We achieve this through a process called quantum feature mapping (also known as *quantum feature embedding* or *data encoding*). We must recall that, at a fundamental level, supervised learning is primarily concerned with the mapping of every *classical feature vector* $\overrightarrow{x} \in \mathbb{R}^n$ into a *quantum state* $|\psi(x)\rangle$ to create complex, non-linear relationships and highlight subtle patterns in the data that are difficult to distinguish in the original classical space by leveraging the higher dimensionality

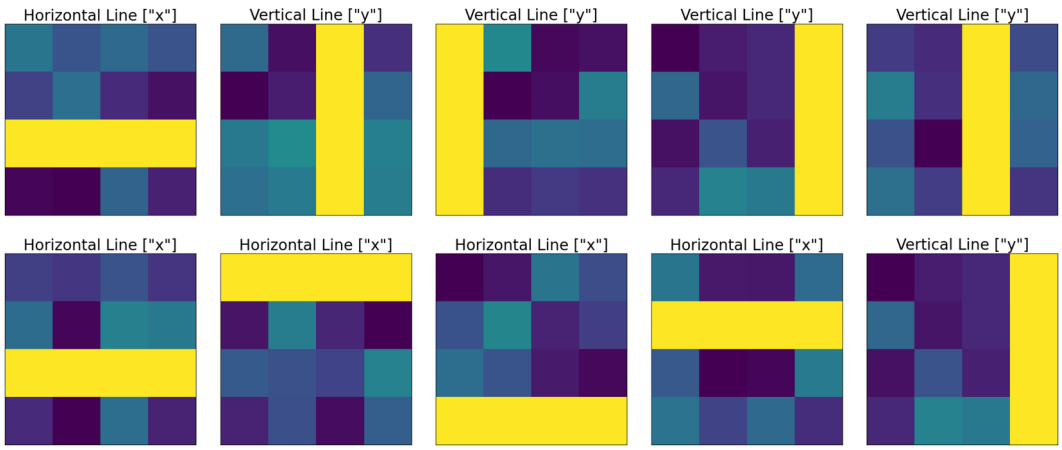


Fig. 2. Reshaped 10 samples of the randomly generated 4×4 binary feature pixelated image dataset with labelled Vertical Line ["y"] and Horizontal Line ["x"]. Each image has a characteristic yellow line pattern with pixel values $\mathrm{q} = \frac{\pi}{2}$ and having a noisy background formed by randomly generating pixel values $\mathrm{p} \in [0, \frac{\pi}{4}]$.

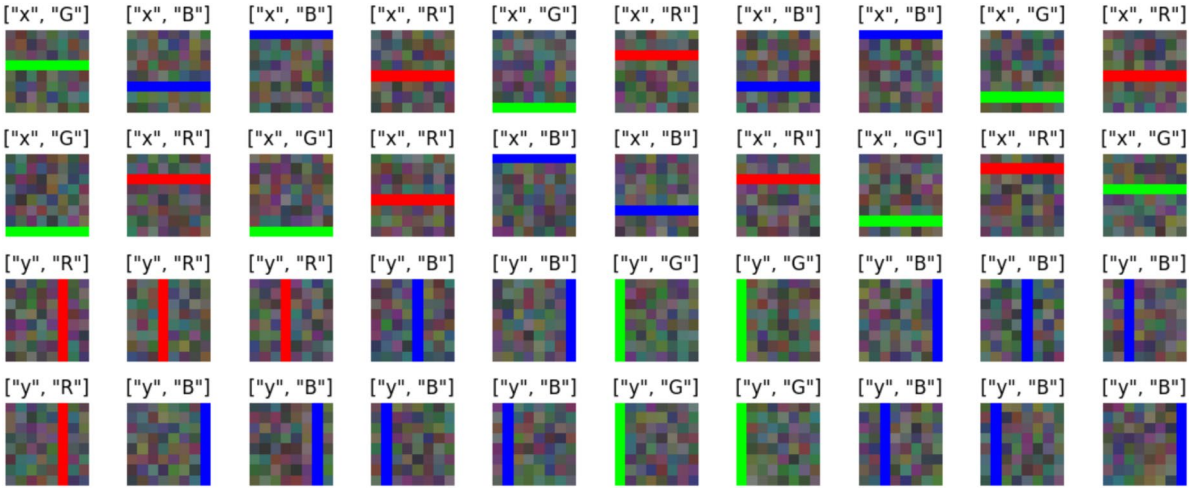


Fig. 3. Reshaped sample of the randomly generated 8 × 8 multiclass feature pixelated image dataset with labelled vertical and horizontal lines assigned with a randomly selected pixel value q associated with each image line colour feature red (q = [255,0, 0]), green (q = [0,255,0]), or blue (q = [0,0, 255]). Every image sample has a noisy background with RGB pixel value p, whose values range between [0,0, 0] and [127,127,127], hence exhibiting multiclass image features for multiclass classification (i.e., image line orientation and colour).

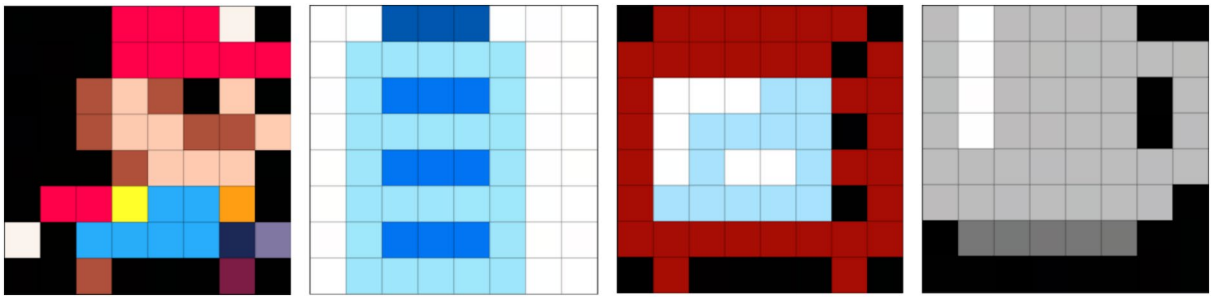


Fig. 4. Four iconised pixelated image samples of the generated 8 × 8 image icon dataset illustrating Mario, Battery, Television, and Cup as read from left to right. Such pixelated image icons can be used in several applications like video games, websites, chat apps, etc. that leverage digital icons with low resolution.

of quantum *Hilbert space* and other inherent properties of quantum states[40]. Thus, feature mapping is important in a way that our parameterised quantum circuit can emulate the exact features and patterns of interest (i.e., features/patterns extracted from the labelled classical image dataset), which form the primary input parameters for the No-QCNN model to better learn relationships among the features. Since different feature maps define different ways of embedding data, we employed the *ZZFeatureMap*, which is a second-order Pauli-Z evolution parameterized quantum circuit that maps classical feature vectors ($\overrightarrow{x}$) into a quantum state ($|\psi(x)\rangle$) by applying layers of *single-qubit rotations* (around the Z-axis of a Bloch sphere) and *two-qubit entangling rotations* that depend on pairs of input features ($x_i x_j$)[41]. The *ZZFeatureMap* also takes in the number of qubits Q and the classical feature vector $\overrightarrow{x}$ as its inputs. It then generates a quantum state $|\psi(x)\rangle$ that represents the data in a high-dimensional quantum feature space. This quantum state, as defined by the parameters of the feature map, is then used as the input of a QML algorithm, such as a *variational quantum classifier* or a *quantum kernel method*, during the training process. It is also important to note that the circuit's structure, not its parameters, is what defines the quantum feature map. Hence, for Q qubits (features), the *ZZFeatureMap* applies the unitary:

$$U_{ZZFeatureMap}\left(\vec{x}\right) = \prod_{\ell=1}^{reps}\left[\left(\bigotimes_{j=1}^{Q} H_j\right)\cdot\left(\prod_{j=1}^{Q} R_Z^{(j)}\left(2x_j\right)\right)\cdot\left(\prod_{i<j} e^{-ix_i x_j Z_i Z_j}\right)\right] \tag{1}$$

where $U_{ZZFeatureMap}$ is the applied unitary gate, $\overrightarrow{x}$ is the classical feature vector such that $x_i x_j$ are pairwise input features, $Z_i Z_j$ represents the tensor product of Pauli-Z operators acting on qubits i and j, H_j is the *Hadamard gate* on the qubit j, $R_Z(2x_j)$ is the data-driven rotation on the qubit j, $e^{-ix_i x_j Z_i Z_j}$ is the entangling ZZ interaction between qubits i and j. By default, the *ZZFeatureMap* circuit also includes *entanglement layers* and

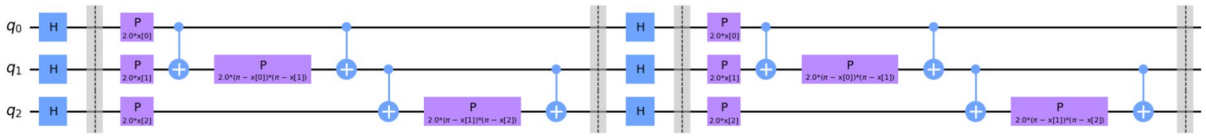


Fig. 5. Illustration of a *ZZFeatureMap* quantum circuit initialised with 3 qubits ($Q = 3$) and having linear entangling layers repeated twice ($reps = 2$).

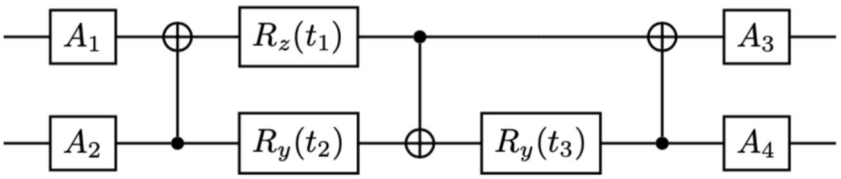


Fig. 6. A sample quantum circuit for implementing a translationally invariant unitary operation of the special unitary group-4 $\{U(4)\}$ with 15 single-qubit and 3 CNOT gates, where t_1, t_2, and t_3 are trainable rotational parameters.

repetition depth (reps), which enables it to perform non-linear mapping, potentially enabling the identification of complex patterns. Figure 5 shows an example of a *ZZFeatureMap* circuit initialised with 3 qubits ($Q = 3$) and having *linear entangling layers* repeated twice ($reps = 2$) between the circuit barriers.

Quantum convolutional and pooling layers

Since we aim to reduce the number of gates and trainable parameters (depth), and the number of qubits (width) of the No-QCNN circuit before performing final qubit measurements, we designed pairs of convolutional and pooling circuit layers, respectively. The convolutional layers were composed of efficient two-qubit translationally invariant quasi-local unitary operators, i.e., special unitary group of degree 4 $\{U(4)\}$ proposed in[34,42,43], and applied on 2-qubit pairs for feature and pattern determination. In theory, it has been shown that an efficient two-qubit quantum gate in $U(4)$, applied on a pair of qubits, requires at most 3 CNOT gates and 15 elementary one-qubit gates from the family $\{R_y, R_z\}$[34]. So, we define every unitary operator $U \in U(4)$ as shown in Eq. 2.

$$U = (A_1 \otimes A_2) \cdot N(\alpha, \beta, \gamma) \cdot (A_3 \otimes A_4) \tag{2}$$

where $A_j \in SU(2)$ special unitary group of degree 2, $\otimes$ is the tensor product of two A_j, $N(\alpha, \beta, \gamma) = e^{i\left(\alpha\sigma_x\sigma_x + \beta\sigma_y\sigma_y + \gamma\sigma_z\sigma_z\right)}$ such that $\alpha, \beta, \gamma \in \mathbb{R}$ are adjustable circuit parameters. Figure 6 shows a sample quantum circuit for implementing the operation of $U(4)$ with three trainable rotational parameters t_1, t_2, and t_3.

The given definition of $U(4)$ implies that each convolutional unitary operation of the No-QCNN would need to have 15 parameters to form a complete basis (for the *Hilbert space*). This means that a pixelated image of $2^n \times 2^n$, the number of parameters becomes very large even with relatively smaller values of n. In essence, it would be very complex and require a lot more training time to fine-tune large numbers of parameters. We combat this issue by firstly adopting the technique proposed in[44], where the quantum circuit ansatz is restricted to a subspace of the *Hilbert space*. Then, our two-qubit unitary gate is defined as $U = N(\alpha, \beta, \gamma)$[34] and applied to neighbouring pairs of qubits in the No-QCNN, resulting in the creation of non-local correlation between two qubits. Then we go an extra step to reduce both the number of trainable parameters and CNOT gates by one-third (1/3) while approximating $U \approx N(\alpha, \gamma)$ using the applicable two-dimensional XXZ quantum Heisenberg model[45–47]. Rather than completely neglecting value of β (by equating $\beta = 0$), we equate it to α (and get $\alpha = \beta \neq \gamma$) such that the Hamiltonian terms are reduced to two-dimensional spin parameters (i.e. $J_{xy} = \alpha = \beta$ and $J_z = \gamma$) where J_{xy} is the isotropic interaction between two neighbouring spins J_x and J_y. Then, we redefine: $N(\alpha, \gamma)$ as (see full derivation of Eq. 3 in Appendix A):

$$N(\alpha, \gamma) = \exp\left[i(\alpha\left(\sigma_x \otimes \sigma_x + \sigma_y \otimes \sigma_y\right) + \gamma\,\sigma_z \otimes \sigma_z)\right] \tag{3}$$

where α and γ are the trainable parameters. So, on one hand $N(\alpha, \gamma)$ reduces the size of our Hilbert space, thereby confining the No-QCNN solutions to a specific subspace that may not even include the optimal solution. On the other hand, it reduces the depth and pairwise gate operational complexity of our quantum circuit, as it has two input parameters, $\alpha$and γ, resulting in a much lesser number of rotational gates. The quantum circuit implementation of $N(\alpha, \gamma)$ represents an efficient unitary operation for the two-qubit pairs in our convolutional layer, as shown in Fig. 7a, with two trainable parameters (i.e. $\theta_1 = 2\gamma - \frac{\pi}{2}$, and $\theta_2 = \frac{\pi}{2} - 2\alpha$) and two CNOT gates. We intentionally eliminated the $R_y\left(2\beta - \frac{\pi}{2}\right)$ gate to avoid unintended operational reversal of the preceding rotational gate $R_y\left(\frac{\pi}{2} - 2\alpha\right)$ since the unitary $U \approx N(\alpha, \gamma)$ is translationally invariant[18]. The

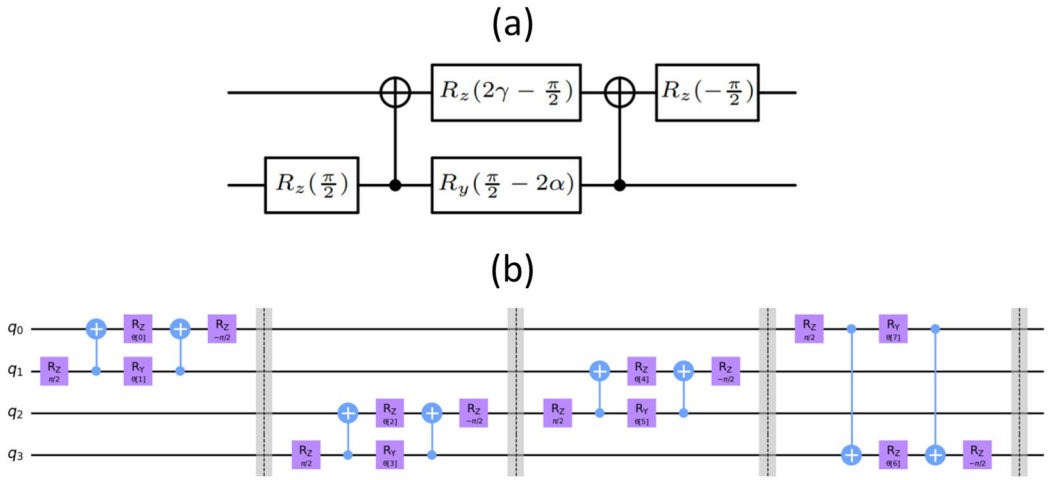


Fig. 7. (**a**) A quantum circuit for implementing a quantum convolutional unitary operation $U \approx N(\alpha, \gamma) = e^{i[\alpha(\sigma_x\sigma_x+\sigma_y\sigma_y)+\gamma\sigma_z\sigma_z]}$ where the two trainable parameters are defined as $\theta_1 = 2\gamma - \frac{\pi}{2}$, and $\theta_2 = \frac{\pi}{2} - 2\alpha$. Note that a global phase, $e^{i(\frac{\pi}{4})}$, is deliberately eliminated. (**b**) A 4-qubit quantum circuit illustrating the implementation of a typical quantum convolutional layer with a generalised convolutional unitary operator [circuit in this figure] applied pairwise on successive qubits having 8 trainable theta parameters $(\theta_0, \theta_1 \cdots \theta_7)$.

designed two-CNOT gate circuit in Fig. 7 is also comparable (but not exact) to the proposed two-CNOT gate implementation of $SO(4)$ in Fig. 2 of[34], which is generally treated as a subset of $U(4)$. This final condensed unitary operator U was applied on a trial 4-qubit quantum circuit for each successive qubit pair to exemplify a complete quantum convolutional layer with 8 trainable theta parameters, as shown in Fig. 7b.

Having a parameterised quantum circuit with optimal gates and trainable parameters is one thing, and reducing the dimensionality (i.e., number of qubits) of the quantum circuit is another. This is where a quantum pooling circuit is needed to shrink the dimensionality of the No-QCNN circuit, thus reducing the number of qubits needed for final measurement. Since this reduction in qubits can't be done in its literal sense, we then define the pooling layer in a somewhat different way from the classical CNNs. To achieve this, we start by counting the number of qubits Q utilised in the preceding No-QCNN convolutional layer. Then we create pairs of the Q qubits and apply a generalised pooling unitary operator {see the circuit in Fig. 8a} that entangles each qubit-pair and copies the quantum information of one qubit onto the other. The qubit that was copied is then ignored for the next operations or measurement. This unitary is applied across all non-repetitive qubit pairs in the circuit depth. The final circuit layer (of the combined unitary operations) has the net effect of encoding the information of two qubits into one, hence reducing the number of qubits needed for the next operations or measurements by half (i.e., $Q/2$). The circuit in Fig. 8b shows a pooling layer formed by applying the generalized unitary operator on a 4-qubit circuit, effectively having 4 theta parameters and forming 2 non-repetitive and unique qubit pairs with qubits q_0 and q_1 (sources) copied into qubits q_2 and q_3 (sinks) respectively, and the sources are ignored for the next circuit operations.

No-QCNN network architecture

The final No-QCNN network architecture was formed by applying quantum convolutional and pooling layers in an alternative sequence on a quantum circuit initialised with the *ZZFeatureMap* and whose number of qubits $Q = 2^n \times 2^n$ for $l = n$ successive convolutional-pooling layers. Figure 9 depicts a typical No-QCNN network architecture encoded with the 4×4 binary feature pixelated image dataset, with the number of qubits $Q = 2^2 \times 2^2 = 16$, such that each qubit $q \in \{q_0, q_1, q_2 \cdots q_{15}\}$, and the number of convolutional-pooling layers $l = 2$. Notice that the measurement is done on the final four qubits q_{12}, q_{13}, q_{14}, and q_{15} (i.e., the final qubit sinks which form the fully connected layer), and a classical register (CR) is used to store measurement results. The fully connected layer is formed with a logarithmically reduced number of qubits ($Q_{fc} = log_2 Q$) and measured after the operation of the nth convolutional-pooling layer, hence, illustrating the diminishing tendency of the No-QCNN network dimensionality due to the pooling unitary operation discussed in Sect. "Quantum convolutional and pooling layers".

Experimental setup

No-QCNN quantum feature map preparation

To set up the experiment, we began by preparing the No-QCNN quantum feature map using the *ZZFeatureMap* in a hierarchical order, starting from image dataset generation and pre-processing all the way through to setting the training and testing data. It is important to emphasise that the No-QCNN employs a *problem-specific quantum feature map*, explicitly designed for low-resolution colour image classification under NISQ constraints, rather than a generic or task-agnostic encoding scheme. The full classical image dataset is taken, and the individual

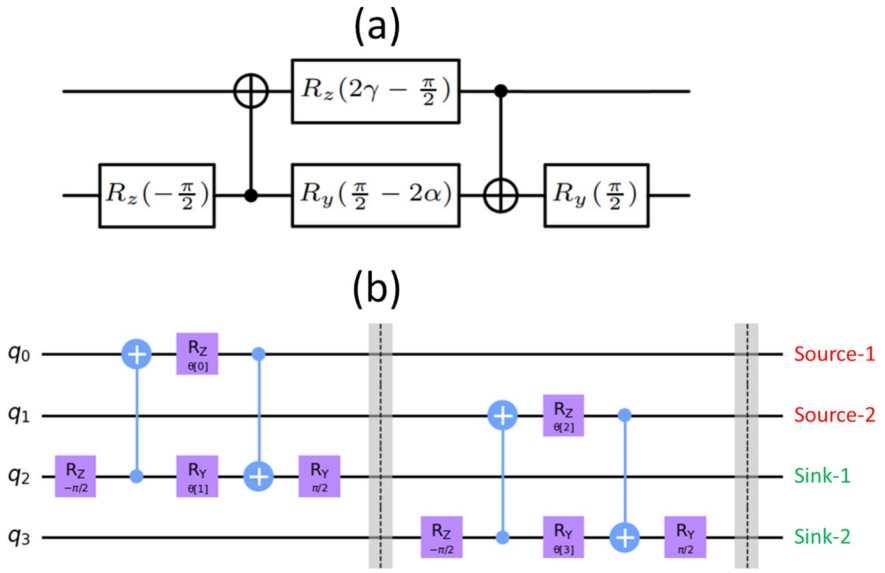


Fig. 8. (**a**) A quantum circuit for implementing a generalised quantum pooling unitary operation. (**b**) A 4-qubit quantum circuit illustrating the implementation of a typical pooling layer with a generalised pooling unitary operator [circuit in this figure] applied pairwise on successive qubits having 4 trainable theta parameters $(\theta_0, \theta_1, \theta_2, \theta_3)$ and forming 2 non-repetitive and unique qubit pairs with qubits q_0 and q_1 (sources) copied into qubits q_2 and q_3 (sinks) respectively. The sources (Source-1 and Source-2) are ignored, and only the sinks (Sink-1 and Sink-2) are utilised in the preceding circuit operations.

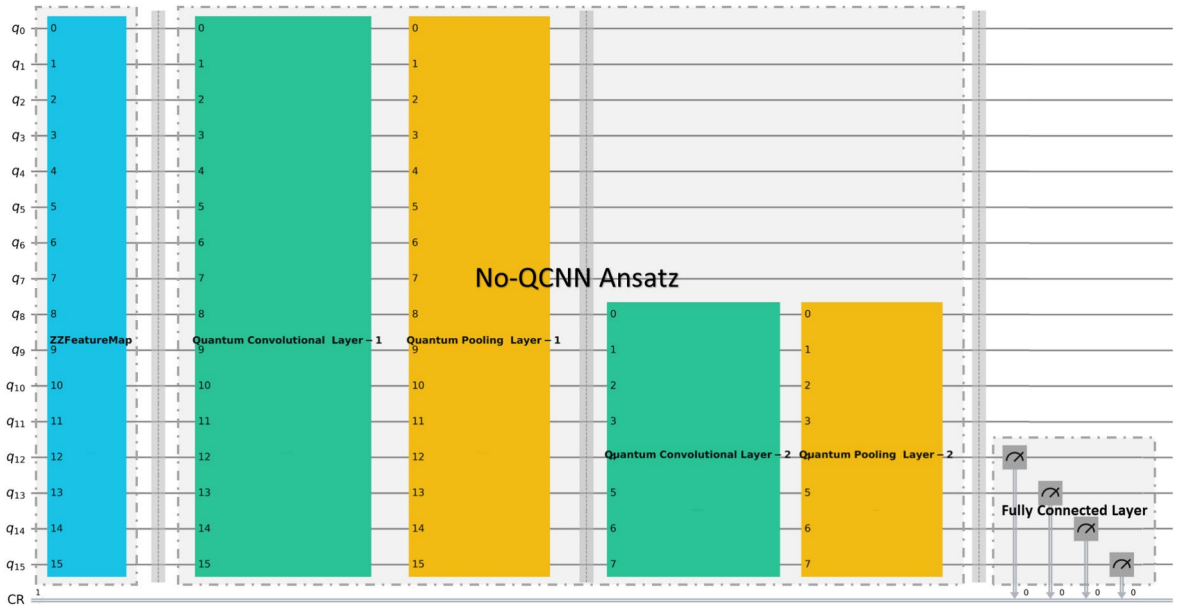


Fig. 9. An example of a typical No-QCNN network architecture encoded with the 4×4 binary feature pixelated image dataset, where the number of qubits $Q = 2^2 \times 2^2 = 16$ and the number of convolutional-pooling layers $l = 2$. Measurement is done on the final four qubits q_{12}, q_{13}, q_{14} and q_{15}, which represents the final qubit sinks that form the fully connected layer ($Q_{fc} = log_2 Q$). A classical register (CR) is used to store the results of measurement in each iteration.

images (e.g., $\text{image}1, 2, \cdots N$) are pre-processed. Each pixel in every $2^n \times 2^n$ square image is represented by a 2-dimensional list, $Pixel(i) = [[R, G, B], [i]]$, where [R, G, B] is the pixel colour combination (intensity values) and $[i] \in \{0,1, \cdots 2^{2n} - 1\}$) is the pixel position. The explicit inclusion of the pixel position alongside colour information ensures that spatial correlations are preserved at the feature-map level, which is not typically the case in standard QCNN encodings that rely solely on pixel intensities. This structured data is rearranged into a

single row block matrix representation having 2^{2n} two-dimensional list elements, with each element representing an image pixel. This means that each block contains the complete feature information (R, G, B, and position) for an entire image. This block-wise organisation constitutes a structured pre-encoding stage that transforms each image into a compact, ordered representation prior to quantum state preparation. Hence, the fully pre-processed image dataset comprises a collection of these blocks that form the 3D dataset block, which is the complete pre-processed dataset. We take note that in the 3D dataset block, two dimensions are taken up by each image (as they have chromatic and spatial data), and the third dimension is the actual image position in the block. This 3D block-matrix representation is a distinctive component of the No-QCNN pipeline and provides a direct interface between classical image structure and quantum encoding. This complete dataset is then divided into two distinct sub-datasets: *Training data* and *Testing data*. The training data was used to train the No-QCNN model, while the testing data was reserved for evaluating the model's performance as it was unseen during training. Finally, the data blocks from the two sub-datasets (representing individual image pixels or features) are converted into quantum states. At this stage, the No-QCNN directly maps the structured 3D block representation into quantum states, thereby embedding spatial and chromatic correlations into the quantum Hilbert space in a unified manner. This process involves encoding the classical feature vector $\overrightarrow{x}$ (R, G, B, and position) into a *Parametrised Quantum Circuit (Quantum Feature Map)* as described in Sect. "Circuit feature mapping (data encoding)". The unique combination of R, G, B, and position values determines the specific rotation angles for a unique set of Pauli rotation operations applied to the circuit's qubits, i.e., $R_x(\phi(x)), R_y(\phi(x)), R_z(\phi(x))$, where x represents the normalised feature values. By construction, this encoding realises a problem-specific quantum feature map that captures joint spatial–chromatic correlations at shallow circuit depth, making it particularly suitable for NISQ-era quantum processors. This embedding process maps the classical feature space into a high-dimensional quantum *Hilbert space*, which is then ready for quantum computation via the No-QCNN network. The complete feature mapping process is illustrated by the pictorial layout in Fig. 10.

Hybrid optimisation for the No-QCNN VQC classifier

The training process begins where the data preparation stage concludes: with the pre-processed image data transformed into quantum feature maps $|\psi_{img}\rangle$. These maps, as established previously, encode the classical pixel information (RGB values and spatial position) into the quantum Hilbert space using a series of parameterised Pauli rotation gates. This initial quantum encoding provides the input state for the *variational quantum classifier* (VQC), which serves as the trainable, quantum core of the No-QCNN, and it is ideal for implementing hybrid quantum–classical optimisation protocols[30,48]. The VQC is a specific type of parameterised quantum circuit, known as the ansatz $U_{ansatz}(\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ represents the array of trainable rotation angles. The ansatz is

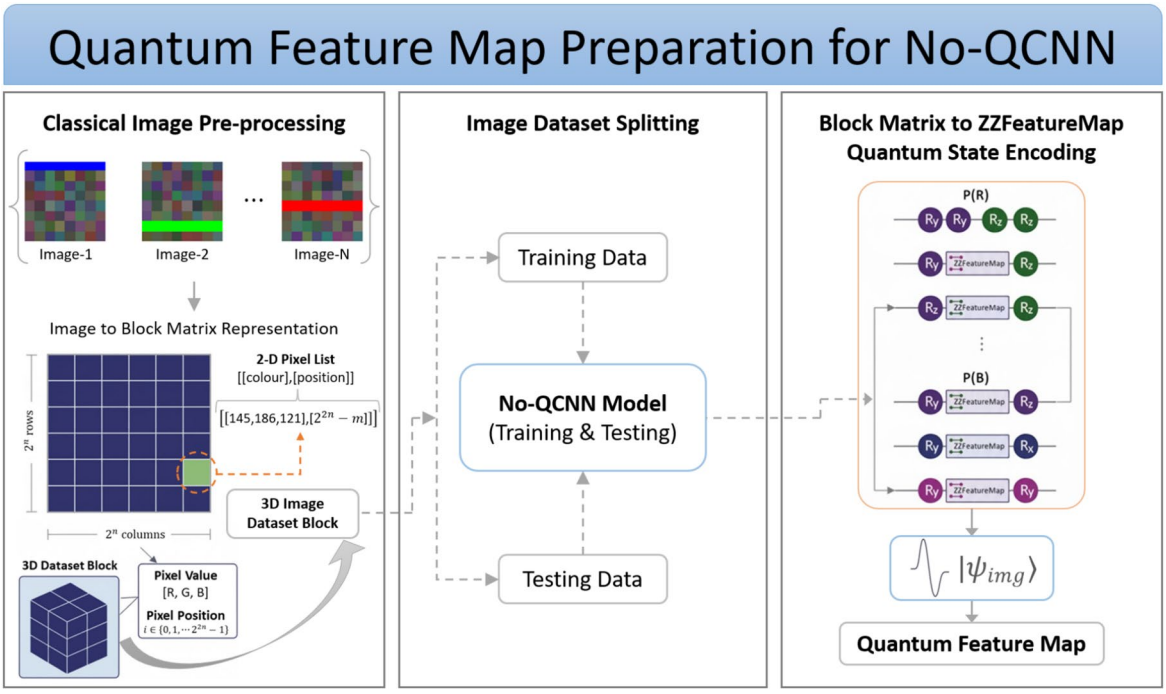


Fig. 10. Schematic illustration of the three-stage pipeline to generate a quantum feature map from classical images for the No-QCNN model. (Left) A dataset of pixelated colour images is transformed into a 3D block, representing all images as a collection of pixels defined by their RGB values and positions, i.e., $Pixel\,(i) = [[R, G, B], [i]]$ where $[R, G, B]$ is the colour combination and $[i] \in \{0,1, \cdots 2^{2n} - 1\}$ is the spatial position. (Centre) This dataset is then systematically split into training and testing subsets. (Right) Data from these subsets is encoded into a quantum state, $|\psi_{img}\rangle$, using the *ZZFeatureMap* to create the final quantum feature map to be used in the No-QCNN mod.

structurally designed to alternate between single-qubit rotations (the parameter tuning elements) and entangling gates (like CNOTs) to introduce necessary quantum correlations. The quantum computation step, which is the QFFNN forward pass, is the application of this ansatz to the feature map as shown in Eq. 4:

$$|\psi(\boldsymbol{\theta})\rangle = U_{\text{ansatz}}(\boldsymbol{\theta}) \; |\psi_{\text{img}}\rangle \tag{4}$$

The output is derived from the expectation value of a measurement observable $\widehat{H}$ (typically $Z^{\otimes N}$) as expressed in Eq. 5, which is estimated statistically by repeatedly running the circuit:

$$\mathcal{E}(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | \widehat{H} | \psi(\boldsymbol{\theta}) \rangle \tag{5}$$

This real-valued expectation $\mathcal{E}(\boldsymbol{\theta})$ constitutes the model's raw prediction score and is subsequently mapped to a classical probability $\widehat{y}$ for classification. The complete training process is a closed-loop system where the classical optimiser continually refines $\boldsymbol{\theta}$ to obtain the optimum value of $\mathcal{E}(\boldsymbol{\theta})$, which is the minimum eigenvalue.

The training objective is formalised as a classical minimisation problem guided by the *Loss Function*, which quantifies the model's performance on the training set. For classification tasks within the No-QCNN framework, the *Cross-Entropy Loss* $\mathcal{L}(\boldsymbol{\theta})$ is the preferred objective function due to its efficacy in dealing with probabilistic outputs $\widehat{y}$ derived from quantum expectation values. The loss for a training set of size N is defined as[49]:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log\left(\widehat{y}_i(\boldsymbol{\theta})\right) + (1 - y_i) \log\left(1 - \widehat{y}_i(\boldsymbol{\theta})\right) \right] \tag{6}$$

In this equation, y_i is the true binary or one-hot encoded label for the i-th data point, and $\widehat{y}_i(\boldsymbol{\theta})$ is the predicted probability derived from $\mathcal{E}_i(\boldsymbol{\theta})$. The core task of the optimisation procedure is to determine the optimal parameter set $\boldsymbol{\theta}^*$ that yields the global minimum of this loss landscape:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) \tag{7}$$

The reliance on probabilistic measurement outcomes introduces *shot noise* into the loss function evaluation. This stochasticity, combined with the computational complexity of estimating the gradient on near-term devices (even with techniques like the Parameter-Shift Rule[50]), necessitates the use of robust *gradient-free optimisation* protocols, setting the stage for the *Constrained Optimisation by Linear Approximations* (COBYLA) algorithm.

COBYLA is the chosen optimiser for the No-QCNN architecture, valued for its stability and efficiency in managing the high-dimensional, noisy, and derivative-free optimisation challenges inherent in VQCs. As a *trust-region method*, COBYLA avoids the explicit, costly, and noise-sensitive computation of gradients, which usually results in barren plateau landscapes[30,51,52]. The algorithm proceeds by iteratively building a local, linear approximation of the objective function $\mathcal{L}(\boldsymbol{\theta}_i)$ based solely on a minimal number of function evaluations within a controlled *trust region*. At each step, COBYLA evaluates the loss at several points (forming a simplex) in the parameter space and constructs a linear model. It then solves a local subproblem to find the next candidate parameter vector $\boldsymbol{\theta}_{i+1}$ that minimises this linear model within the trust region constraints. The iterative mechanism provides a highly effective feedback loop, as shown in Fig. 11.

The closed-loop optimisation process depicted in Fig. 11 comprises four consecutive stages. 1. *Classical Selection*: The COBYLA optimiser generates a candidate parameter vector $\boldsymbol{\theta}_i$; 2. *Quantum Execution*: The generated vector is transmitted to the quantum processor (or simulator) to parameterise the *VQC* $\{U_{ansatz}(\boldsymbol{\theta}_i)\}$. The circuit is executed repeatedly with the quantum feature map $|\psi_{img}\rangle$ to obtain the expectation value $\mathcal{E}(\boldsymbol{\theta}_i) = \langle \psi(\boldsymbol{\theta}_i) | H | \psi(\boldsymbol{\theta}_i) \rangle$; 3. *Loss Evaluation*: The classical processor then computes the corresponding loss function $\mathcal{L}(\boldsymbol{\theta}_i)$; and 4. *Parameter Update*: The scalar loss value is subsequently returned to COBYLA, which uses this information to update its internal model, adjust the trust region size, and generate the next parameter estimate $\boldsymbol{\theta}_{i+1}$ (until it reaches the most optimal parameter $\boldsymbol{\theta}^*$), thereby closing the optimisation loop and driving the system toward a minimum loss. This robust gradient-free approach significantly reduces the quantum resource overhead; fewer quantum circuit calls are required compared to gradient-based methods, making the overall training of the No-QCNN both theoretically sound and experimentally viable on contemporary NISQ hardware. Once training is complete, the final, optimised parameters $\boldsymbol{\theta}^*$ are fixed and used to evaluate the model's performance on the unseen testing data (which also enters the model via the same quantum feature mapping procedure described in Sect. "No-QCNN quantum feature map preparation") to ensure generalisation and guard against overfitting.

Results and discussion

This section presents a comprehensive empirical evaluation of the No-QCNN framework. The experiments were conducted entirely within the IBM Qiskit SDK by employing the *Qiskit Machine Learning* library's VQC classifier[53] within a virtual Python environment. We utilised a local quantum simulator (i.e., *Qiskit Estimator* and *Sampler*) to establish a noise-free performance baseline. Our analysis is structured into three parts. First, we conduct a deep dive into the training dynamics and classification performance of the No-QCNN on the binary dataset to distinguish horizontal versus vertical yellow lines. Second, we extend this analysis to a more complex multiclass classification problem involving both line orientation and colour combinations, probing the model's ability to learn a richer feature set. Finally, we contextualise the performance of our quantum-inspired model by

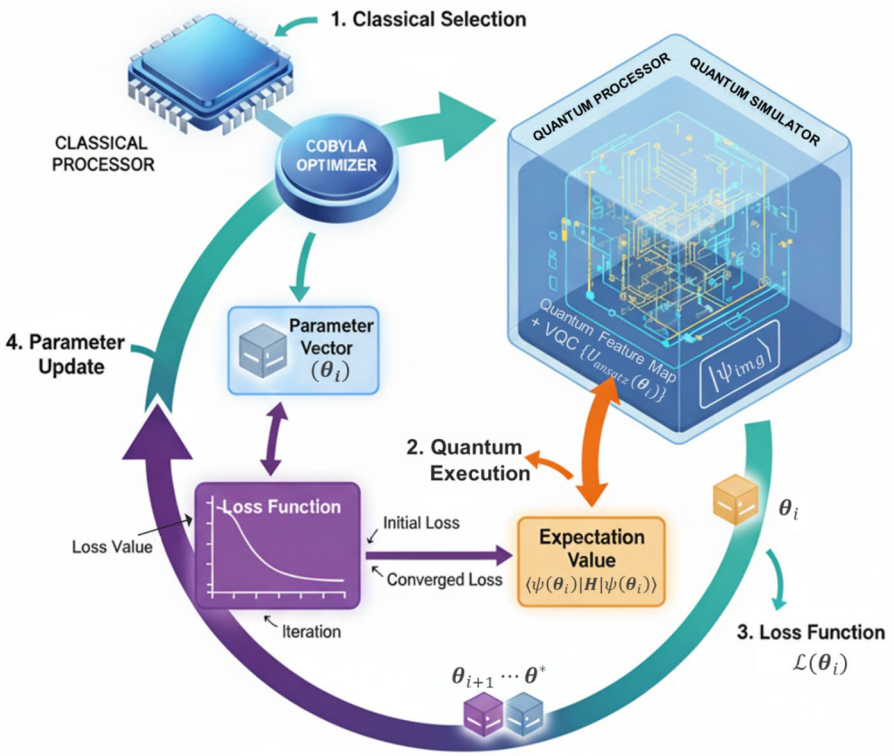


Fig. 11. Iterative optimisation process of the No-QCNN variational quantum classifier (VQC) using the COBYLA algorithm. The diagram illustrates the four consecutive stages of the closed-loop optimisation cycle. 1. Classical Selection: The COBYLA optimiser (a classical processor) initiates the cycle by generating a new candidate parameter vector ($\boldsymbol{\theta}_i$). 2. Quantum Execution: This vector ($\boldsymbol{\theta}_i$) is transmitted to the Quantum Processor/Simulator to parameterise the VQC $\{U_{ansatz}(\boldsymbol{\theta}_i)\}$, which includes the Quantum Feature Quantum Map ($|\psi_{img}\rangle$). The circuit is repeatedly executed to obtain the expectation value of the measurement operator, $\langle\psi(\boldsymbol{\theta}_i)|H|\psi(\boldsymbol{\theta}_i)\rangle$. 3. Loss Evaluation: The classical processor receives the expectation value and uses it to compute the scalar loss function $\mathcal{L}(\boldsymbol{\theta}_i)$. 4. Parameter Update: The scalar loss value is then returned to the COBYLA optimiser, which uses this feedback to update its internal model, adjust the trust region size, and generate the next parameter estimate $\boldsymbol{\theta}_{i+1}$ (until it reaches the most optimal parameter $\boldsymbol{\theta}^*$), thereby closing the optimisation loop and driving the system toward a minimum loss.

benchmarking it against a standard classical CNN, enabling a detailed discussion on the potential, challenges, and appropriate application domains for this class of QML algorithms.

No-QCNN binary classification

To evaluate the performance of the proposed No-QCNN architecture, we implemented a binary classification task based on synthetic image data generated as 4×4 pixelated colour grids (with a total of 16 features). Each image corresponded to one of two classes: *vertical yellow lines* or *horizontal yellow lines* embedded within randomly coloured backgrounds. The use of yellow line patterns was intentional, as it ensured clear pixel-level contrast between feature (signal) and noise (background) elements, thereby allowing the quantum classifier to exploit feature correlations through entanglement and rotational encoding. The dataset construction process, described in Sect. "Dataset generation and preprocessing", produced distinct yet statistically balanced sets of 10, 25, 50, 75, and 100 training images, which were used to examine the scalability and generalisation behaviour of the No-QCNN. The training and validation datasets were split at 70:30 for all classification tasks. The variational circuit embedded the trainable parameters of the No-QCNN feature map and the convolutional-pooling layer pairs, effectively mimicking convolutional receptive fields in classical CNNs[54]. The optimisation of variational parameters was performed using the gradient-free COBYLA algorithm under a hybrid quantum–classical loop, where the objective function minimised the *cross-entropy loss* between predicted and true class labels.

The convergence behaviour and optimisation trends of the No-QCNN model are illustrated and summarised in Fig. 12 as observed across five datasets with 10, 25, 50, 75, and 100 image samples. Figure 12a shows the objective function convergence with increasing iteration. For smaller datasets (10–25 images), the objective function exhibited a noticeable decrease, indicative of stable and noise-resilient optimisation. On the other hand, the larger datasets (50, 75, and 100 images) stagnated to minimise the objective values almost throughout the iteration loop, while smaller datasets demonstrated a slower but more refined minimisation, which is consistent with an increased and a reduced dimensional search space, respectively. The gradual but steady decrease of the objective curve for smaller data volumes (10 and 25 images) reflects a smoother loss landscape achieved through

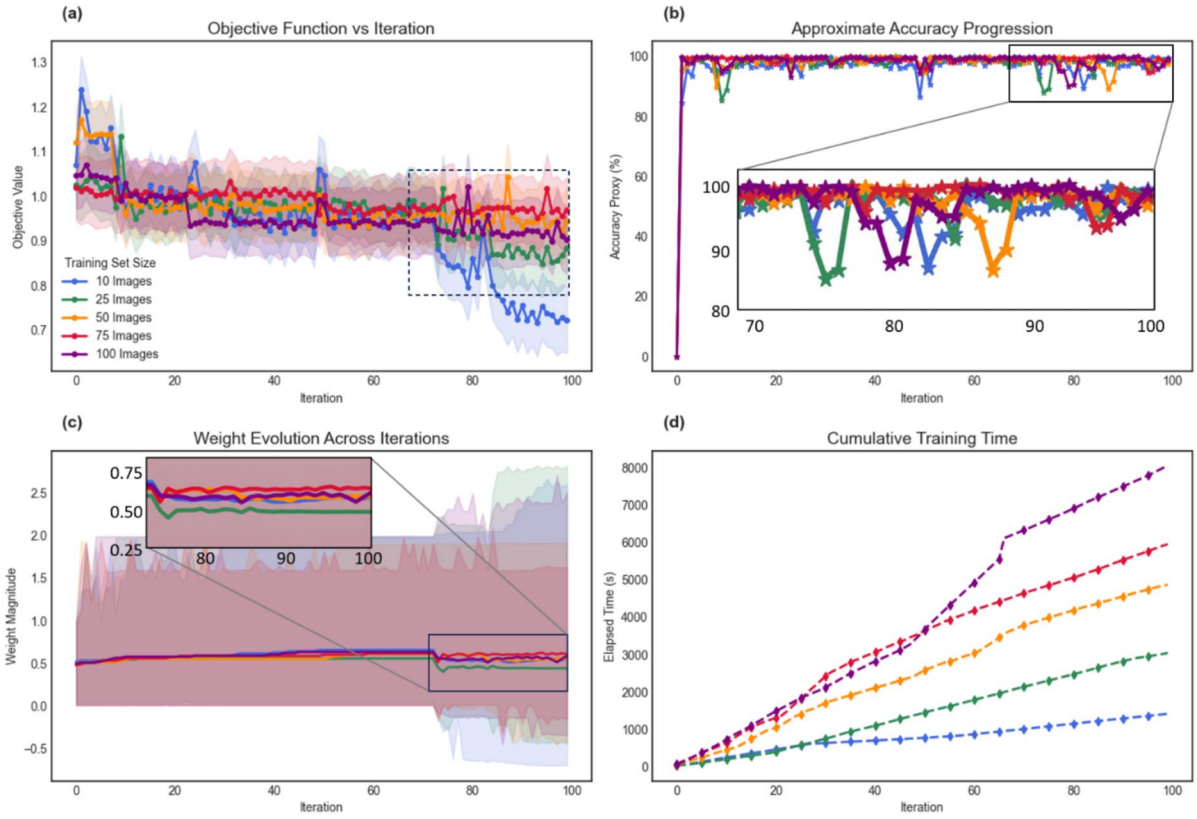


Fig. 12. The plots illustrate the hybrid quantum–classical optimisation and training progression of the proposed No-QCNN for datasets containing 10, 25, 50, 75, and 100 binary image samples. (**a**) Objective function values had a noticeable decrease for smaller datasets, thus showing a stable and noise-resilient optimisation. The larger datasets stagnated almost throughout the iteration loop due to an increased dimensional search space. (**b**) Approximate training accuracy exhibits nearly the same level of saturation for all datasets, which reflects a somewhat steady training progression of the VQC. (**c**) The evolution of trainable circuit weights shows bounded parameter oscillations and a fairly steady min–max spread across all datasets towards the end, suggesting a successful minimisation of the variational optimisation landscape. (**d**) The cumulative training time scales linearly with dataset size due to increased circuit evaluations in the classical data handling loop (as the number of feature/qubits remained constant for the generated image sizes of 4×4 pixels, i.e., the number of qubits $Q = 16$ for the No-QCNN feature map). Collectively, the results demonstrate the feasibility and learning stability of the No-QCNN framework under hybrid variational optimisation on binary image data.

Dataset size (Images)	Training accuracy (%)	Validation ACCURACY (%)	Training iterations	Total training time (s)
10	85.71	93.22	100	1406.61
25	88.82	97.45	100	3035.34
50	73.14	89.70	100	4866.43
75	65.54	84.95	100	5950.43
100	61.86	78.83	100	8055.24

Table 1. Summary of training and validation metrics for binary classification of the No-QCNN model for the first 100 iterations across image datasets of different sizes.

entanglement-mediated dropout regularisation[55]. The inset in Fig. 12a further highlights a distinct transition region around iteration 80, particularly visible in the 10-image dataset where the objective value drops sharply while variance tightens for larger datasets. This discontinuity marks the phase where the COBYLA optimizer contracts its trust region radius, shifting the search strategy from broader exploration to fine-grained exploitation of the local minimum. Figure 12b presents the approximate accuracy progression for each dataset. The No-QCNN achieved nearly the same saturation levels for all datasets, which reflects a somewhat steady training generalisation of the VQC. However, the final training accuracies decreased significantly with an increase in dataset size (beyond 25 images) within the first 100 iterations, as shown in Table 1. This pattern suggests

that the model's quantum feature mapping did not scale favourably with data complexity, but the observed approximate accuracy progression confirms that the variational parameters reached stable quasi-optimal regions in the parameter space. As highlighted by the zoomed-in view around iteration 75 to 90 in Fig. 12b, small but persistent accuracy dips are observed for larger datasets, reflecting sensitivity to stochastic optimiser steps and finite-sampling effects rather than divergence or instability. The narrowing spread of accuracy fluctuations beyond this point further supports the conclusion that the training dynamics have stabilised despite reduced expressive capacity at higher data volumes.

In Fig. 12c, the evolution of average, minimum, and maximum circuit weights is shown as a function of iteration. Across all dataset sizes, the weights initially fluctuated strongly with sharp peaks and flat valleys as the optimiser explored the variational landscape, but gradually stabilised into a steady distribution band with minimal fluctuations. The steady min–max spread of weights with successive iterations is a hallmark of variational convergence, suggesting that the hybrid training procedure successfully minimised redundant parameter oscillations and localised around stable minima. This stabilisation also demonstrates that the parameterised ansatz employed in the No-QCNN was expressive enough to capture the binary image features without requiring excessive circuit depth. The inset in Fig. 12c captures a critical phase transition starting from iteration ~ 75. Unlike the earlier iterations where parameters were tightly clustered, this region reveals a sudden expansion and downward shift in the weight distribution. This divergence aligns with the sharp drop in the objective function, indicating that the optimizer successfully escaped a restrictive local minimum and diversified the circuit parameters to access a deeper, more optimal region of the cost landscape. Finally, Fig. 12d illustrates the cumulative training time as a function of iteration for each dataset. The elapsed time increased linearly with an increase in dataset size (see Table 1), an observation consistent with reported results in[56,57], reflecting the additional circuit evaluations required by COBYLA during each iteration. This scaling trend means that the primary bottleneck in our VQC training process was the classical data handling loop, not the complexity of the quantum computation per data point, since each No-QCNN feature map had the same number of qubits/features for the generated images with sizes 4×4 pixels (i.e. the number of qubits $Q = 16$) even though each dataset had a different number of images. The linear growth associated with the data handling loops reaffirms the computational feasibility of No-QCNN training under classical supervision for small-to-moderate qubit systems. However, it's crucial to remember that this does not mean the VQC is linear in general. If you were to increase the number of features (which increases the number of qubits) or the number of variational parameters, you would still see the underlying polynomial scaling for that "Constant Time Block."

The results from the No-QCNN binary classification experiment are summarised in Table 1 and serve as an indicator for a successful proof-of-concept. They validate that the No-QCNN architecture, with its quantum feature map and convolutional-pooling variational layers, is fundamentally trainable and can solve a simple, linearly separable problem in classical binary image data. As we moved from the dataset with 10 to 25 images, the final training and validation accuracy improved slightly, with the highest training accuracy at 88.82% and a corresponding validation accuracy at 97.45% obtained on the dataset with 25 images. A drastic drop was noticed as well when we moved from datasets with 50 to 100 images, where the training and validation accuracy of 61.86% and 78.83% respectively, were observed for a dataset with 100 images. The lower final training and validation accuracies for larger datasets, however, demonstrate that the model could not reliably scale up with the current configuration.

No-QCNN multiclass classification

To probe the limits of the No-QCNN's feature extraction capabilities, we scaled the problem complexity to a six-class classification task. The dataset for this experiment consisted of images containing either a horizontal or a vertical line, where the line could be one of three colours: Red, Green, or Blue (see Fig. 3), thus, effectively creating a multiclass image dataset with 6 classes as discussed in Sect. "Dataset generation and preprocessing". This required the model not only to learn orientation but also to discriminate between colour channels, effectively learning a composite feature vector. We generated three image datasets having 50, 75, and 100 image samples of pixel sizes 8×8, thus having $Q = 64$ features (one should note that the multiclass dataset sizes started from 50 to avoid null image representation for both training and testing datasets in each of the 6 classes, and a training to validation dataset split ratio of 70:30 was maintained). However, we used the RGB bit shifting method such that each [R, G, B] colour pixel value was converted to a unique integer value as shown in Eq. (8):

$$\text{Pixel Value}_{\{[R,G,B]\}} = \left(R \cdot 2^{16}\right) + \left(G \cdot 2^{8}\right) + B \tag{8}$$

Then we applied the *Principal Component Analysis* (PCA)[58,59], as a dimensionality reduction method to reduce a 64-feature image block matrix to 16 features. In this way, the No-QCNN model architecture, including the number of qubits and circuit depth, was kept constant to facilitate a direct comparison of the problem's impact on a fixed-resource model since only the number of classes had changed from 2 (binary) to 6 (multiclass).

The final convergence behaviour and optimisation trends of the No-QCNN model on the three multiclass image datasets are shown in Fig. 13, which basically shows a fairly similar trend of results to the one in Fig. 12. In Fig. 13a, the objective function values for the dataset with 50 images were greatly minimised compared to the ones with 75 and 100 images. This is indicative of a stagnated optimisation process that may not have achieved a local minimum within the first 100 iterations on the larger datasets. As observed in binary classification, the approximate accuracy saturation levels were the same on average as depicted in Fig. 13b. The min–max spread of weights, as shown in Fig. 13c, starts more steadily for the first 25–30 iterations but fluctuates a lot more towards the last 25 iterations. This, however, shows that the multiclass optimisation process was not as efficient and reliable, and also that the model training was not expressive enough to correctly learn all 6 image features in the three training datasets. In the last panel, i.e., Fig. 13d, we see that the total training times for the multiclass

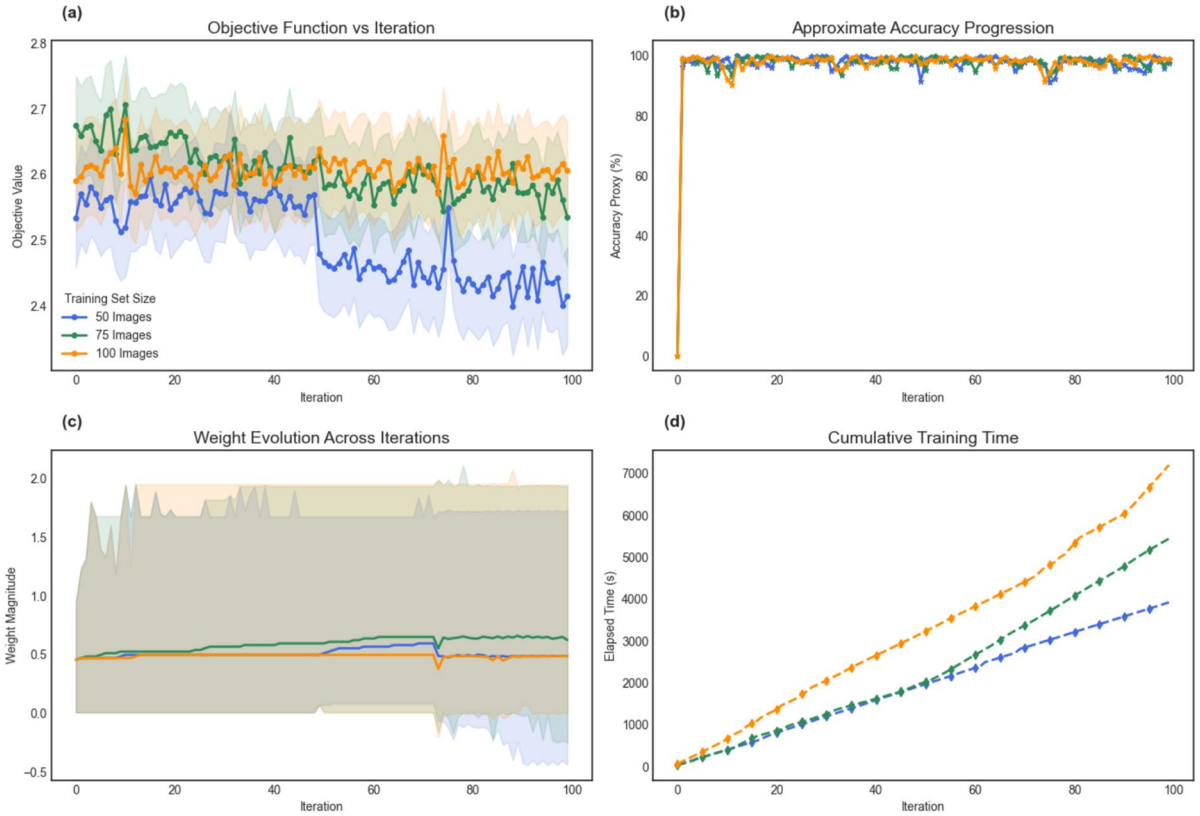


Fig. 13. The plots illustrate the hybrid quantum–classical optimisation and training progression of the proposed No-QCNN for datasets containing 50, 75, and 100 multiclass images distributed across 6 classes. (**a**) Objective function values decrease gradually for the dataset with 50 images and stagnate for the rest, indicative of an optimisation process with non-local minima. (**b**) Approximate training accuracy exhibits nearly the same level of saturation for all datasets, which reflects a somewhat steady training progression of the VQC. (**c**) The evolution of weights starts more steadily for the first 25–30 iterations but fluctuates a lot more towards the last 25 iterations which shows that the multiclass optimization and training process was not as efficient and reliable to correctly learn all the 6 features of the training datasets (**d**) The cumulative training time scales linearly with dataset size due to increased circuit evaluations in the classical data handling loop (as the number of feature/qubits remained constant for the generated images of size 8×8 pixels, and further reduced using PCA to 8×8 i.e., the final $Q = 16$ for the No-QCNN feature map). The results demonstrate the potential of No-QCNN to scale for multiclass classification, even though the current configuration could not learn reliably to improve the training/validation accuracy on multiclass datasets.

Dataset size (Images)	Training accuracy (%)	Validation accuracy (%)	Training iterations	Total training time (s)
50	66.20	82.05	100	3917.64
75	59.38	78.19	100	5439.16
100	53.29	65.77	100	7201.04

Table 2. Summary of training and validation metrics for multiclass classification of the No-QCNN model for the first 100 iterations across image datasets of different sizes, each having 6 classes.

datasets were considerably lower compared to the training times of similar binary dataset sizes (compare total training times in Tables 1 and 2 for datasets with 50, 75, and 100 image samples). This meant that, given the same number of features and dataset size, the No-QCNN model was quicker with training 6 classes than it was with 2 classes of data. We attribute this to the quantum parallelism, which speeds up the process since the data sizes in per class, for multiclass datasets, were much lower. A comparable observation was reported in[60], where, in their case, a QCNN model outperformed a classical CNN on a dataset with 6 classes. And as we saw earlier with binary classification, the total training times for multiclass classification also increased as the image datasets increased (a result attributed to the overload on the classical data handling loop).

Table 2 summarises the training and validation performance of the No-QCNN on six-class image classification tasks as the dataset size increases. A clear trend is observed: both training and validation accuracies decrease as the number of images grows from 50 to 100, while the total training time increases substantially.

Classification task	Machine learning model	Training accuracy (%)	Validation accuracy (%)	Training time (s) [Iterations/Epochs]
Binary	No-QCNN	73.14	89.7	4866.43 (100 iter)
Binary	CNN	100	100	4.72 (50 epochs)
Multiclass	No-QCNN	66.20	82.05	3917.64 (100 iter)
Multiclass	CNN	82.86	40.00	4.36 (50 epochs)

Table 3. Comparative performance metrics of the proposed No-QCNN and a classical CNN on binary and multiclass image datasets comprising 50 image samples distributed across 2 and 6 classes, respectively.

This behaviour reflects fundamental constraints of NISQ-compatible quantum learning rather than deficiencies of the proposed architecture. For the smallest dataset (50 images), the No-QCNN achieves a relatively modest training accuracy of 66.20% but a significantly higher validation accuracy of 82.05%. This gap indicates strong generalisation and suggests that the unitary constraints and shallow-depth variational circuit act as an implicit regulariser, preventing overfitting in low-data regimes. Such behaviour contrasts with classical CNNs, which typically exhibit high training accuracy but degraded validation performance under similar conditions.

Taken together, the results in Tables 1 and 2 define the effective operating regime and current scalability limits of the No-QCNN framework under NISQ constraints[21]. In the binary classification task (Table 1), the model demonstrates reliable trainability and strong generalisation for small datasets, validating its proof-of-concept capability; however, a systematic decline in performance emerges as dataset size increases, indicating limited expressive capacity within the present shallow-depth configuration[38,51]. This trend becomes more pronounced in the multiclass setting (Table 2), where increased class complexity further amplifies the capacity-data mismatch imposed by fixed quantum feature maps, restricted circuit depth, and finite sampling. Overall, these results indicate that No-QCNN is best suited to low-data learning scenarios, particularly multiclass problems with strong correlation structure, where its quantum feature extraction and implicit regularisation can offer improved generalisation. The observed performance degradation with increasing data scale is therefore consistent with known NISQ limitations and underscores the need for more expressive encodings and deeper yet trainable circuits as quantum hardware continues to mature[66,67].

No-QCNN vs. classical CNN: a comparative analysis

Having simulated the No-QCNN model, it was essential to benchmark and properly contextualise its performance against a relevant classical counterpart. As we know, for image classification, the gold standard is the classical CNN[61–63]. So, we designed a simple CNN (using *TensorFlow* platform) with a complexity roughly analogous to our No-QCNN, consisting of two convolutional layers followed by a dense layer (see Fig. 1 in Sect. "Background"). The CNN model was trained on both the binary and multiclass datasets, and the final comparative results with the No-QCNN model across both classification tasks for the binary and multiclass image datasets (both with dataset sizes of 50 images) are summarised in Table 3 (see the classical CNN training results in Appendix B).

Table 3 compares the performance of the proposed No-QCNN with a classical CNN on both binary and multiclass image classification tasks using small datasets of 50 images each. The comparison focuses on three key metrics: training accuracy, validation accuracy, and total training time, providing insights into the efficiency and generalisation capability of both models. For the binary classification task, the classical CNN achieved 100% accuracy on both the training and validation sets after 50 epochs, indicating full convergence and excellent performance on this small dataset. The training time was approximately 4.72 s, demonstrating the computational efficiency of classical architectures on small-scale data. In contrast, the No-QCNN achieved a slightly lower training accuracy of 73.14% but a higher validation accuracy of 89.7% after 100 optimisation iterations. Despite its longer training time of 4866.43 s, the quantum-inspired model exhibited superior generalisation ability, suggesting that its VQC structure may capture more robust features and avoid overfitting compared to the classical CNN, as it has very high expressivity[64].

In the multiclass classification task, the No-QCNN again demonstrated a strong generalisation advantage, attaining a validation accuracy of 82.05%, compared to only 40.00% for the classical CNN. Although the classical model achieved a higher training accuracy (82.86%), its poor validation performance indicates overfitting and limited capability to generalise beyond the training samples. The No-QCNN, on the other hand, maintained balanced training and validation accuracies (66.20% and 82.05%, respectively), reflecting its ability to model complex multiclass relationships even with limited data. However, this advantage came at the expense of significantly longer training times (~ 3917.64 s) due to the computational overhead of quantum circuit evaluations and parameter optimisation. Overall, Table 3 highlights a fundamental trade-off between computational cost and generalisation capability. While classical CNNs train rapidly and achieve perfect fitting on small binary datasets, the proposed No-QCNN demonstrates superior generalisation performance, particularly for multiclass classification tasks.

However, it is important to note that the proposed No-QCNN does not entirely aim to replace classical CNNs in large-scale data regimes, but rather addresses a well-defined computational niche where classical CNNs are prone to overfitting and inefficiency, namely, low-data, high-correlation tasks. The innovation of No-QCNN lies in (i) the use of structured quantum feature maps that encode both chromatic and spatial correlations at shallow circuit depth, (ii) quantum convolution and pooling operations that reduce circuit width while preserving essential correlations all the way through to the fully connected qubit layer (see Fig. 9), and (iii) a variational quantum classifier that acts as an implicit regulariser due to unitary constraints. Overall, the results discussed

in Sect. "Results and discussion" should be understood as the validation of the potential of quantum-enhanced models that can capture richer data representations and mitigate overfitting in limited-data regimes. This, ultimately, paves the way for more efficient hybrid quantum–classical approaches in image classification tasks.

Conclusion

This study introduced the Novel Quantum Convolutional Neural Network (No-QCNN), a hybrid quantum–classical learning model that merges variational quantum circuits (VQC) with convolution-pooling inspired feature extraction to enhance image classification performance. Implemented using IBM's local quantum simulator and utilising COBYLA gradient-free optimiser, the No-QCNN model was successfully trained and validated on binary and multiclass datasets with {10, 25, 50, 75, and 100}, and {50, 75, and 100} images, respectively. The No-QCNN achieved a superior validation accuracy of 82.05% for multiclass classification tasks on small image datasets comprising 50 images (across 6 classes), thus, outperforming a classical CNN benchmark that exhibited strong overfitting and a validation accuracy of 40.00% despite its rapid convergence. As for the training accuracy, evaluated on the same validation dataset, the CNN model performed extremely well (82.86%) compared to the No-QCNN (66.20%). In the simpler binary task, the classical CNN was way more superior, achieving a perfect 100% accuracy against the No-QCNN's 89.7% after the first 50 epochs and 100 iterations, respectively. The results demonstrate that quantum circuit-based learning can yield more robust generalisation even in low-data regimes where classical deep networks typically underperform. These findings highlight the growing potential of quantum neural architectures to expand the landscape of efficient and noise-resilient learning. The extended training time observed in the No-QCNN, as compared to the CNN, reflects present limitations in quantum circuit execution and simulator throughput rather than the model's inefficiency. As quantum processors evolve with improved qubit coherence, lower gate errors, and more efficient parameter-optimisation protocols, the computational gap between classical and quantum models will progressively narrow. The No-QCNN framework, therefore, represents a forward-looking paradigm that aligns closely with the expected trajectory of near-term quantum hardware development.

Looking ahead, future work will aim at improving the current configuration of the No-QCNN model to enhance its expressibility and extend its application to larger and more complex image datasets. We will also test its scalability on real quantum processors and explore adaptive circuit designs that optimise depth, entanglement, and learning stability. The integration of noise-aware optimisation, quantum error mitigation, and dynamic ansatz construction may further enhance performance and interpretability. Beyond image recognition, the principles demonstrated here are broadly applicable to scientific imaging, medical diagnostics, and quantum data analysis, where hybrid architectures could deliver meaningful computational advantages. Ultimately, the No-QCNN marks a critical step toward realising quantum-enhanced intelligence systems capable of matching and/or surpassing classical learning limits. Its successful implementation affirms that the path toward practical quantum advantage in machine learning is both achievable and imminent, establishing the foundation for the next generation of intelligent computing and quantum-enhanced artificial vision systems powered by quantum technologies.

Data availability

The data used in this research can be found in the shared Google Drive link below: (https://drive.google.com/drive/folders/1sE6wGadWATFkG6mqQdYOhGIHw0F-yHsA).

Appendix

Appendix A: Derivation of two-parameter XXZ quantum heisenberg model—$N\ (\alpha, \gamma)$

1.Notation and preliminaries.

We use the standard Pauli matrices as follows:

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \tag{A1}$$

and the computational two-qubit basis $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$ (ordered in that sequence). Tensor products follow the usual Kronecker convention, and the following simple facts will be used repeatedly:

$$\sigma_x^2 = \sigma_y^2 = \sigma_z^2 = I \tag{A2}$$

And for X a 2×2 Pauli matrix,

$$e^{-i\theta X} = \cos\theta I + i\sin\theta X (\text{since } X^2 = I \tag{A3}$$

2.Matrix form of the generator $H\ (\alpha, \beta, \gamma)$

We start by computing the four 4×4 operators $\sigma_\mu \otimes \sigma_\mu$ in the computational basis in the computational basis and form the linear combination. A direct calculation (or standard Kronecker product algebra) gives:

$$H(\alpha,\beta,\gamma)=\begin{pmatrix}\gamma & 0 & 0 & \alpha-\beta\\ 0 & -\gamma & \alpha+\beta & 0\\ 0 & \alpha+\beta & -\gamma & 0\\ \alpha-\beta & 0 & 0 & \gamma\end{pmatrix} \tag{A4}$$

where rows/columns are indexed 00, 01, 10, 11. (You can expand the Kronecker products directly to verify this.) Observe the block structure: the subspaces {|00⟩, |11⟩} and {|01⟩, |10⟩} are invariant because no matrix elements connect those pairs to the other pair. This observation already simplifies the problem: we only need to exponentiate two 2×2 blocks. This block decomposition is a direct linear-algebraic fact; the same block structure is used repeatedly in spin Hamiltonian treatments of the XXZ model[45,46]. Thus, we write the two blocks explicitly as follows:

- For even subspace $E = span\{|00\rangle, |11\rangle\}$where $X=\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ (i.e., *Pauli x* in the subspace).

$$B_{\text{even}}=\begin{pmatrix}\gamma & \alpha-\beta\\ \alpha-\beta & \gamma\end{pmatrix}=\gamma I_2+(\alpha-\beta)X \tag{A5}$$

- And for odd subspace $O = span\{|01\rangle, |10\rangle\}$

$$B_{\text{odd}}=\begin{pmatrix}-\gamma & \alpha+\beta\\ \alpha+\beta & -\gamma\end{pmatrix}=-\gamma I_2+(\alpha+\beta)X \tag{A6}$$

Both blocks are of the simple form $cI + dX$, and since $X^2 = I$ they are straightforward to exponentiate as shown in the next section.

3.Exponentiation of the 2×2 blocks (closed forms).

For any real scalars c, d and $X^2 = I$, and using the Baker–Campbell–Hausdorff (BCH) formula for commutating operators (i.e. $e^{A+B}=e^Ae^B$), the exponential satisfies:

$$e^{i(cI+dX)}=e^{icI}e^{idX}=e^{ic}I(\cos d\,I+i\sin d\,X)=e^{ic}(\cos d\,I+i\sin d\,X) \tag{A7}$$

Then we apply Eq. A7 to the two blocks as follows:

- For the even block, set $c=\gamma, d=\alpha-\beta$. Then,

$$e^{\,iB_{\text{even}}}=e^{\,i\gamma}(\cos(\alpha-\beta)\,I+i\sin(\alpha-\beta)\,X) \tag{A8}$$

- For the odd block, set $c=-\gamma, d=\alpha+\beta$. Then

$$e^{\,iB_{\text{odd}}}=e^{-i\gamma}(\cos(\alpha+\beta)\,I+i\sin(\alpha+\beta)\,X) \tag{A9}$$

Placing these 2×2 block exponentials back into the 4×4 computational basis yields the explicit closed-form for $N(\alpha,\beta,\gamma)=e^{iH}$:

$$N(\alpha,\beta,\gamma)=\begin{pmatrix}e^{i\gamma}\cos(\alpha-\beta) & 0 & 0 & i\,e^{i\gamma}\sin(\alpha-\beta)\\ 0 & e^{-i\gamma}\cos(\alpha+\beta) & i\,e^{-i\gamma}\sin(\alpha+\beta) & 0\\ 0 & i\,e^{-i\gamma}\sin(\alpha+\beta) & e^{-i\gamma}\cos(\alpha+\beta) & 0\\ i\,e^{i\gamma}\sin(\alpha-\beta) & 0 & 0 & e^{i\gamma}\cos(\alpha-\beta)\end{pmatrix} \tag{A10}$$

The above equation reduces to the special forms often encountered in quantum-information contexts. Specifically, identical block decompositions arise in the standard treatment of a spin chain interaction described by the anisotropic Heisenberg XXZ Hamiltonian with nearest-neighbour interactions[47].

4.The XXZ two-parameter specialisation: set $\beta=\alpha$

Here we set $\beta=\alpha=a$ and let $b=\gamma$. Then:

- $\alpha-\beta=0\Rightarrow$ the even block becomes γI, hence

$$e^{iB_{even}}=e^{i\gamma}\,I_2 \tag{A11}$$there are no off-diagonal transitions between $|00\rangle$ and $|11\rangle$.

- $\alpha+\beta=2a\Rightarrow$ the old block becomes

$$e^{iB_{odd}}=e^{-i\gamma}\,(\cos(2a)\,I+i\sin(2a)\,X) \tag{A12}$$

So, the full unitary is:

$$N(a,b) = \begin{pmatrix} e^{ib} & 0 & 0 & 0 \\ 0 & e^{-ib}\cos(2a) & i\,e^{-ib}\sin(2a) & 0 \\ 0 & i\,e^{-ib}\sin(2a) & e^{-ib}\cos(2a) & 0 \\ 0 & 0 & 0 & e^{ib} \end{pmatrix} \tag{A13}$$

This is the explicit two-parameter XXZ family unitary $N(a,b)$ associated with the two-qubit XXZ Hamiltonian.

$$H_{XXZ}(a,b) = a(\sigma_x \otimes \sigma_x + \sigma_y \otimes \sigma_y) + b\,\sigma_z \otimes \sigma_z \tag{A14}$$

Hence, physically, in many-body notation, we can write $J_{xy} = a$, $J_z = b$, and the anisotropy parameter is $\Delta = J_z/J_{xy}$. In our case, we neglect the anisotropic parameter and assume that $J_{xy} = \alpha$ and $J_z = \gamma$, giving us the final two-parameter unitary operator $N(\alpha,\beta)$ defined as:

$$N(\alpha,\gamma) = e^{\,iH_{XXZ}} = \exp\left[i\left(\alpha\left(\sigma_x \otimes \sigma_x + \sigma_y \otimes \sigma_y\right) + \gamma\,\sigma_z \otimes \sigma_z\right)\right] \tag{A15}$$

Appendix B: Results of classical CNN training on image datasets with 50 images

To establish a reference baseline for the proposed No-QCNN, a classical convolutional neural network (CNN) was trained on image datasets of 50 samples. Figures 14 and 15 illustrate the training dynamics for binary and multiclass classification tasks, respectively, showing consistent convergence of both training and validation metrics across 25, 50, 75, and 100 epochs. Key summaries of the training outcomes, including final accuracies and losses, are reported in Tables 4 and 5, highlighting the network's ability to effectively learn from small-scale datasets. These results provide a clear benchmark for subsequent comparison with the No-QCNN (see Sect. "No-QCNN vs. classical CNN: a comparative analysis"), emphasising both the strengths and limitations of conventional CNN performance on limited image data.

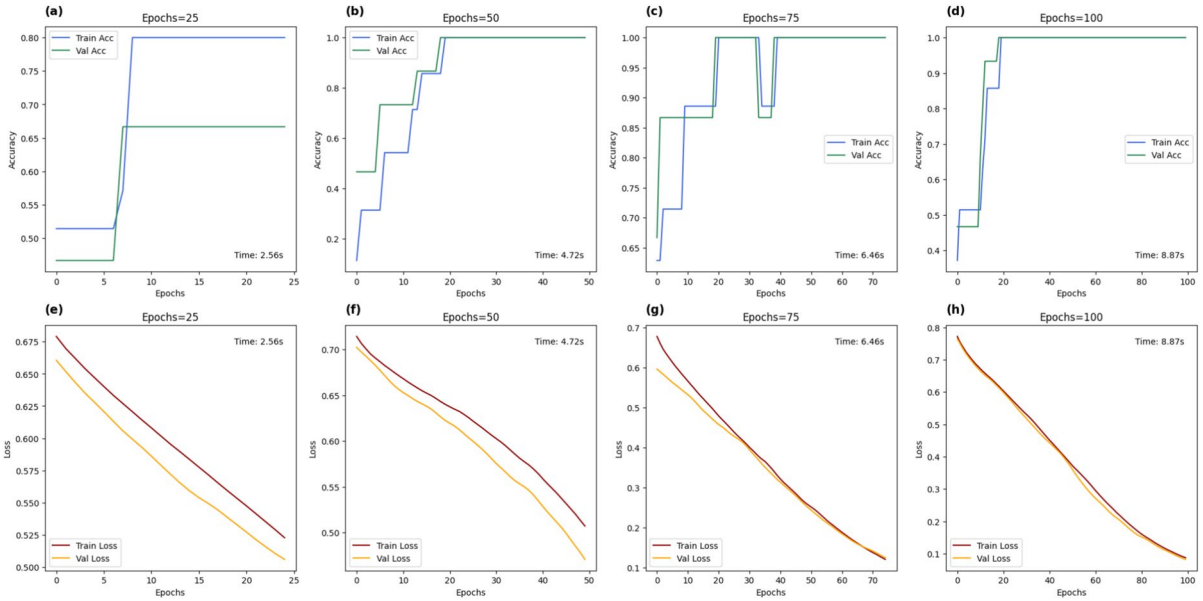


Fig. 14. Training and validation performance of the CNN on a binary dataset with 50 images across different epoch settings. Plots (**a**)–(**d**) show the evolution of training accuracy (blue) and validation accuracy (green) for 25, 50, 75, and 100 training epochs, respectively. Plots (**e**)–(**h**) display the corresponding training loss (dark red) and validation loss (orange) curves for the same epoch sets. Each subplot reports the total wall-clock training time (in seconds) in the bottom-right corner. The results demonstrate that increasing the number of training epochs generally improves both training and validation accuracy while reducing loss, though diminishing returns are observed at higher epochs. Labels (**a**)–(**h**) correspond to subplots for clarity, highlighting convergence trends for accuracy and loss metrics in this simple CNN benchmark against the No-QCNN model.

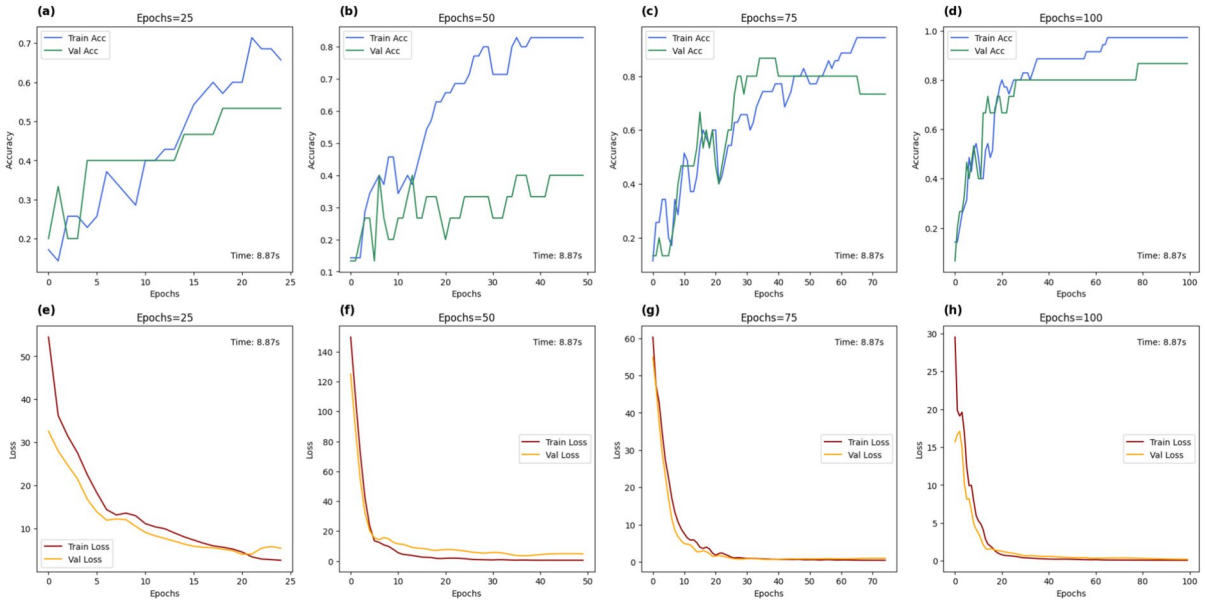


Fig. 15. Illustration of the accuracy and loss metrics for a multiclass classification model trained on a dataset of 50 images across 6 classes. The performance is evaluated over four separate training runs with a varying number of epochs: (**a**), (**e**) 25 epochs, (**b**), (**f**) 50 epochs, (**c**), (**g**) 75 epochs, and (**d**), (**h**) 100 epochs. The top row (**a**)–(**d**) displays the training accuracy (blue) versus validation accuracy (green). The bottom row (**e**)–(**h**) displays the training loss (brown) versus validation loss (orange). Execution times are noted for each run.

Epochs	Final train accuracy (%)	Final validation accuracy (%)	Train loss	Validation loss	Total training time (s)
25	80.00	66.67	0.5227	0.5059	2.56
50	100.00	100.00	0.5073	0.4708	4.72
75	100.00	100.00	0.1207	0.1252	6.46
100	100.00	100.00	0.0873	0.0823	8.87

Table 4. Classical CNN performance metrics across varying epoch counts for the binary dataset with 50 images.

Epochs	Final train accuracy (%)	Final validation accuracy (%)	Train loss	Validation loss	Total training time (s)
25	65.71	53.33	2.6075	5.4089	2.66
50	82.86	40.00	0.5036	4.8004	4.36
75	94.29	73.33	0.4470	0.9918	6.01
100	97.14	86.67	0.0584	0.1876	8.42

Table 5. Classical CNN performance metrics across varying epoch counts for a multiclass dataset with 50 images, each having 6 classes.

The results in Fig. 14 demonstrate that the CNN achieved rapid convergence and strong generalisation even with a small binary dataset of only 50 images. At 25 epochs (Fig. 14a, e), the training accuracy reached ~ 80% and validation accuracy ~ 67%, with moderately high loss values, suggesting the model was still learning low-level representations. As the number of epochs increased to 50 (Fig. 14b, f), both training and validation accuracies reached 100%, indicating perfect classification performance, while the losses dropped sharply, signifying efficient parameter optimisation and model stability. Further extending training to 75 and 100 epochs (Fig. 14c–d, g–h) maintained perfect accuracy while slightly reducing both training and validation losses, reaching 0.0873 and 0.0823 at 100 epochs. This steady decline in loss values despite stable accuracies reflects improved confidence and convergence of the CNN model. The close alignment of training and validation curves across all epoch settings confirms that the model generalised exceptionally well without overfitting, a notable achievement given the small dataset size. Additionally, training time increased nearly linearly from 2.56 s at 25 epochs to 8.87 s at 100 epochs, reflecting computational scalability. Overall, Fig. 14 highlights the

CNN's capacity to effectively learn binary class distinctions from a limited dataset, achieving full convergence and stability between 50 and 100 epochs, with consistent validation performance and minimal loss divergence. The results presented in Table 4 show a consistent improvement in the CNN's performance as the number of training epochs increased from 25 to 100. At 25 epochs, the model achieved a training accuracy of 80.00% and a validation accuracy of 66.67%, with moderately high loss values, indicating partial learning and limited generalisation. As training continued to 50 epochs, both training and validation accuracies reached 100.00%, signifying complete convergence of the network and effective feature extraction from the small 4×4 input images. Further training at 75 and 100 epochs maintained perfect classification accuracy while progressively reducing both training and validation losses, reaching 0.0873 and 0.0823, respectively, at 100 epochs. This steady decline in loss values reflects the CNN's increasing confidence and stability in predictions with extended training. Notably, the training time scaled nearly linearly with epoch count, from 2.56 s at 25 epochs to 8.87 s at 100 epochs, demonstrating efficient computational performance. Overall, these findings confirm that the CNN effectively learned the underlying data distribution and achieved optimal generalisation within 50–100 epochs, with negligible overfitting.

The empirical results, depicted in Fig. 15, reveal a critical relationship between training duration and the CNN model's ability to generalise from the training data to unseen validation samples. During initial training phases of 25 and 50 epochs, the model demonstrated a rapid capacity to learn the training dataset, as evidenced by the monotonically increasing training accuracy and decreasing training loss (Fig. 15a, b, e, f). However, the corresponding validation metrics indicated poor generalisation. The validation accuracy exhibited erratic behaviour and plateaued at suboptimal levels, while a notable divergence between the training and validation loss curves signalled the onset of overfitting, particularly in the 50-epoch trial, where validation loss began to stagnate. This disparity suggests that the model was beginning to memorise idiosyncrasies of the training set rather than learning robust, generalizable features. Upon extending the training duration to 75 and subsequently 100 epochs, a marked improvement in model generalisation was observed. While the training accuracy continued its ascent towards unity, the validation accuracy also climbed significantly, ultimately reaching a stable and high-performance plateau at ~ 80% (Fig. 15d). This convergence was mirrored in the loss dynamics, where the validation loss successfully minimized and stabilized at a low value, closely tracking the trajectory of the training loss without significant divergence (Fig. 15g, h). This behaviour indicates that extended training enabled the model to move beyond fitting noise and converge on a more effective representation of the underlying data structure, thereby enhancing its predictive power on the validation set. The total computational time scaled linearly with the number of epochs, increasing from 2.66 s to 8.42 s. Collectively, these findings underscore that a sufficient training duration of at least 75 epochs is crucial for mitigating overfitting and achieving a robust, high-performing classification model for this dataset, establishing a clear trade-off between computational cost and generalisation performance.

The results presented in Table 5 demonstrate the progressive improvement in the classical CNN's performance with an increasing number of training epochs. At 25 epochs, the model exhibited moderate learning with a training accuracy of 65.71% and validation accuracy of 53.33%, accompanied by relatively high loss values, indicating underfitting. Extending the training to 50 epochs improved the training accuracy to 82.86%, though the validation accuracy dropped to 40.00%, suggesting possible overfitting at this stage. A more balanced performance emerged at 75 epochs, where both training and validation accuracies increased to 94.29% and 73.33%, respectively, with a significant reduction in validation loss. The optimal performance was achieved at 100 epochs, yielding the highest training and validation accuracies of 97.14% and 86.67%, alongside the lowest loss values (0.0584 and 0.1876). This trend indicates that prolonged training enabled the model to generalise better and converge effectively, albeit at the cost of increased computational time. Overall, the results suggest that 100 epochs provided the best trade-off between accuracy, loss reduction, and training efficiency.

Appendix C: Research data repository (source code and data files)

The source code and data used in this research can be found in the public GitHub repository link below: https://github.com/TGCHOMIE/NoQCNN-Project

Received: 27 November 2025; Accepted: 17 March 2026
Published online: 27 March 2026

References

1. Wiley, V. & Lucas, T. Computer vision and image processing: A paper review. *Int. J. Artif. Intell. Res.* **2**(1), 22. https://doi.org/10.29099/ijair.v2i1.42 (2018).
2. Qu, G., Song, Q. & Fang, T. The artistic image processing for visual healing in smart city. *Sci Rep* **14**, 16846. https://doi.org/10.1038/s41598-024-68082-7 (2024).
3. Wang, Y. Application of improved image processing technology in new media short video quality improvement in film and television postproduction. *Int. J. Digit. Multimed. Broadcast.* **2023**, 1–9. https://doi.org/10.1155/2023/5579242 (2023).
4. Prabaharan, T., Periasamy, P., Mugendiran, V., & Ramanan. (2020). Studies on application of image processing in various fields: An overview.In: *IOP Conference Series. Materials Science and Engineering, 961, 012006*. https://iopscience.iop.org/article/https://doi.org/10.1088/1757-899X/961/1/012006
5. Mahmood, Z. Digital image processing: Advanced technologies and applications. *Appl. Sci.* **14**(14), 6051. https://doi.org/10.3390/app14146051 (2024).
6. Chidziwisano, G., Samikwa, E. & Daka, C. Deep learning methods for poultry disease prediction using images. *Comput. Electron. Agric.* **230**, 109765. https://doi.org/10.1016/j.compag.2024.109765 (2025).
7. O'Shea, K., & Nash, R. (2015). An introduction to Convolutional Neural Networks. arXiv.org. https://doi.org/10.48550/arXiv.1511.08458
8. Yamashita, R. et al. Convolutional neural networks: An overview and application in radiology. *Insights Imaging* **9**, 611–629. https://doi.org/10.1007/s13244-018-0639-9 (2018).

9. Aquino-Brítez, S., García-Sánchez, P., Ortiz, A. & Aquino-Brítez, D. Towards an energy consumption index for deep learning models: A comparative analysis of architectures, GPUs, and measurement tools. *Sensors* **25**(3), 846. https://doi.org/10.3390/s25030846 (2025).
10. Mebtouche, N. E. D. & Sahnoune, S. Quantum Computing for Computer Vision: Applications, Challenges, and Research Tracks. In *Quantum Computing: Applications and Challenges QSAC 2023 Information Systems Engineering and Management* (eds Drias, H. & Yalaoui, F.) (Springer, Cham, 2024).
11. Gill, S. S., *et al.* (2024, September 6). Quantum Computing: Vision and challenges. arXiv.org. https://doi.org/10.48550/arXiv.2403.02240
12. Pfaendler, S. M. L., Konson, K. & Greinert, F. Advancements in quantum computing—Viewpoint: Building adoption and competency in industry. *Datenbank Spektrum* **24**, 5–20. https://doi.org/10.1007/s13222-024-00467-4 (2024).
13. La Cour, B. Advances in quantum computing. *Entropy* **25**(12), 1633. https://doi.org/10.3390/e25121633 (2023).
14. Senokosov, A. et al. Quantum machine learning for image classification. *Mach. Learn. Sci. Technol.* **5**(1), 015040. https://doi.org/10.1088/2632-2153/ad2aef (2024).
15. Li, Q., Huang, Z., Yang, H., & Song, M. (2023). A review of Quantum Machine Learning for Image Classification. In: 2023 International Conference on Intelligent Management and Software Engineering (IMSE), 70–75. https://doi.org/10.1109/imse61332.2023.00021
16. Biamonte, J. et al. Quantum machine learning. *Nature* **549**, 195–202. https://doi.org/10.1038/nature23474 (2017).
17. Peral-García, D., Cruz-Benito, J. & García-Peñalvo, F. J. Systematic literature review: Quantum machine learning and its applications. *Comput. Sci. Rev.* **51**, 100619. https://doi.org/10.1016/j.cosrev.2024.100619 (2024).
18. Cong, I., Choi, S. & Lukin, M. D. Quantum convolutional neural networks. *Nat. Phys.* **15**, 1273–1278. https://doi.org/10.1038/s41567-019-0648-8 (2019).
19. Gong, L.-H., Pei, J.-J., Zhang, T.-F. & Zhou, N.-R. Quantum convolutional neural network based on variational quantum circuits. *Opt. Commun.* **550**, 129993. https://doi.org/10.1016/j.optcom.2023.129993 (2024).
20. Song, Y. et al. A resource-efficient quantum convolutional neural network. *Front. Phys.* **12**, 1362690. https://doi.org/10.3389/fphy.2024.1362690 (2024).
21. Cerezo, M. et al. Variational quantum algorithms. *Nat. Rev. Phys.* **3**, 625–644. https://doi.org/10.1038/s42254-021-00348-9 (2021).
22. Moll, N. et al. Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Sci. Technol.* **3**(3), 030503. https://doi.org/10.1088/2058-9565/aab822 (2018).
23. Gujju, Y., Matsuo, A., & Raymond, R. (2024). Quantum machine learning on near-term quantum devices: Current state of supervised and unsupervised techniques for real-world applications. Physical Review Applied, **21**(6). https://doi.org/10.1103/physrevapplied.21.067001
24. Sun, Y. et al. Scalable quantum convolutional neural network for image classification. *Phys. A Stat. Mech. Its Appl.* **657**, 130226. https://doi.org/10.1016/j.physa.2024.130226 (2025).
25. Wei, S. et al. A quantum convolutional neural network on NISQ devices. *AAPPS Bull.* **32**, 2. https://doi.org/10.1007/s43673-021-00030-3 (2022).
26. Yu, X., Fernando, B., Hartley, R., & Porikli, F. (2018). Super-resolving very low-resolution face images with supplementary attributes. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 908–917. https://doi.org/10.1109/cvpr.2018.00101
27. Zhang, L. et al. Making low-resolution satellite images reborn: A deep learning approach for super-resolution building extraction. *Remote Sens.* **13**(15), 2872. https://doi.org/10.3390/rs13152872 (2021).
28. Seo, J. & Park, H. Object recognition in very low resolution images using deep collaborative learning. *IEEE Access* **7**, 134071–134082. https://doi.org/10.1109/access.2019.2941005 (2019).
29. Janowczyk, A. et al. A resolution adaptive deep hierarchical (RADHicaL) learning scheme applied to nuclear segmentation of digital pathology images. *Comput. Methods Biomech. Biomed. Eng. Imaging Vis.* **6**(3), 270–276. https://doi.org/10.1080/21681163.2016.1141063 (2016).
30. Daka, C. & Bhattacharyya, S. Quantum neural network-inspired variational quantum circuit for simulating diamond 14NV center hamiltonian with a proximal 13C isotope. *Phys. Scr.* **100**(2), 025111. https://doi.org/10.1088/1402-4896/ada58d (2025).
31. Chen, G. et al. Quantum convolutional neural network for image classification. *Pattern Anal. Appl.* **26**, 655–667. https://doi.org/10.1007/s10044-022-01113-z (2023).
32. Zheng, J. et al. Design of a quantum convolutional neural network on quantum circuits. *J. Frankl. Inst.* **360**(17), 13761–13777. https://doi.org/10.1016/j.jfranklin.2022.07.033 (2023).
33. Chen, S. et al. Quantum convolutional neural networks for High Energy Physics data analysis. *Phys. Rev. Res.* https://doi.org/10.1103/physrevresearch.4.013231 (2022).
34. Vatan, F. & Williams, C. Optimal quantum circuits for general two-qubit gates. *Phys. Rev. A* https://doi.org/10.1103/physreva.69.032315 (2004).
35. Kraus, B. & Cirac, J. I. Optimal creation of entanglement using a two-qubit gate. *Phys. Rev. A*. https://doi.org/10.1103/physreva.63.062309 (2001).
36. Luna-Hernandez, S., Quintana, C., & Rosas-Ortiz, O. (2024). Optimizing entanglement in two-qubit systems. arXiv.org. https://arxiv.org/abs/2407.18474
37. Kundu, A. et al. Enhancing variational quantum state diagonalization using reinforcement learning techniques. *New J. Phys.* **26**(1), 013034. https://doi.org/10.1088/1367-2630/ad1b7f (2024).
38. Hu, F. et al. Overcoming the coherence time barrier in quantum machine learning on temporal data. *Nat. Commun.* **15**, 7491. https://doi.org/10.1038/s41467-024-51162-7 (2024).
39. Gujju, Y., Matsuo, A. & Raymond, R. Quantum machine learning on near-term quantum devices: Current state of supervised and unsupervised techniques for real-world applications. *Phys. Rev. Appl.* https://doi.org/10.1103/physrevapplied.21.067001 (2024).
40. Havlíček, V. et al. Supervised learning with quantum-enhanced feature spaces. *Nature* **567**, 209–212. https://doi.org/10.1038/s41586-019-0980-2 (2019).
41. IBM Quantum. (2025). ZZFeatureMap (latest version). IBM Quantum Documentation. https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.ZZFeatureMap
42. Schuch, N. & Siewert, J. Natural two-qubit gate for quantum computation using the XY interaction. *Phys. Rev. A*. https://doi.org/10.1103/physreva.67.032301 (2003).
43. Chen, Z. et al. Efficient implementation of arbitrary two-qubit gates using unified control. *Nat. Phys.* **21**, 1489–1496. https://doi.org/10.1038/s41567-025-02990-x (2025).
44. Qiskit-Community. (2024). Qiskit Machine Learning. GitHub. https://github.com/qiskit-community/qiskit-machine-learning/blob/stable/0.8/docs/tutorials/11_quantum_convolutional_neural_networks.ipynb
45. Fehér, G. & Pozsgay, B. The propagator of the finite XXZ spin-1/2 chain. *SciPost Phys.* **6**(5), 063. https://doi.org/10.21468/scipostphys.6.5.063 (2019).
46. Pushpan, C. B. et al. Estimating correlations and entanglement in the two-dimensional Heisenberg model in the strong-rung-coupling limit. *Phys. Rev. A* **110**(3), 032408. https://doi.org/10.1103/physreva.110.032408 (2024).
47. Serra, P., Ferrón, A. & Osenda, O. Almost perfect quantum state transfer on isotropic and anisotropic Heisenberg spin chains with tailored site dependent exchange couplings. *Phys. Lett. A.* **449**, 128362. https://doi.org/10.1016/j.physleta.2022.128362 (2022).
48. Long, C. et al. Hybrid quantum-classical-quantum convolutional neural networks. *Sci. Rep.* **15**, 31780. https://doi.org/10.1038/s41598-025-13417-1 (2025).

49. Xiang, Q. et al. Quantum classical hybrid convolutional neural networks for breast cancer diagnosis. *Sci. Rep.* **14**, 24699. https://doi.org/10.1038/s41598-024-74778-7 (2024).
50. Wierichs, D., Izaac, J., Wang, C. & Lin, C.Y.-Y. General parameter-shift rules for quantum gradients. *Quantum* **6**, 677. https://doi.org/10.22331/q-2022-03-30-677 (2022).
51. McClean, J. R. et al. Barren plateaus in quantum neural network training landscapes. *Nat. Commun.* **9**, 4812. https://doi.org/10.1038/s41467-018- (2018).
52. Arrasmith, A., Cerezo, M., Czarnik, P., Cincio, L. & Coles, P. J. Effect of barren plateaus on gradient-free optimization. *Quantum* **5**, 558. https://doi.org/10.22331/q-2021-10-05-558 (2021).
53. Qiskit Ecosystem. (2018). VQC - Qiskit Machine Learning 0.8.4. https://qiskit-community.github.io/qiskit-machine-learning/stubs/qiskit_machine_learning.algorithms.VQC.html#
54. Zhao, X. et al. A review of convolutional neural networks in computer vision. *Artif. Intell. Rev.* **57**, 99. https://doi.org/10.1007/s10462-024-10721-6 (2024).
55. Kobayashi, M., Nakaji, K. & Yamamoto, N. Overfitting in quantum machine learning and entangling dropout. *Quantum Mach. Intell.* **4**, 30. https://doi.org/10.1007/s42484-022-00087-9 (2022).
56. Haug, T., Self, C. N. & Kim, M. S. Quantum machine learning of large datasets using randomized measurements. *Mach. Learn. Sci. Technol.* **4**(1), 015005. https://doi.org/10.1088/2632-2153/acb0b4 (2023).
57. Cho, G. & Kim, D. Machine learning on quantum experimental data toward solving quantum many-body problems. *Nat. Commun.* **15**, 7552. https://doi.org/10.1038/s41467-024-51932-3 (2024).
58. Pearson, K. On lines and planes of closest fit to systems of points in space. *Philos. Mag.* **2**(11), 559–572 (1901).
59. Hotelling, H. Analysis of a complex of statistical variables into principal components. *J. Educ. Psychol.* **24**(6), 417–441. https://doi.org/10.1037/h0071325 (1933).
60. Mordacci, M., Ferrari, D., Amoretti, M. (2024). Multi-class quantum convolutional Neural Networks. arXiv.org. https://doi.org/10.48550/arXiv.2404.12741
61. Krizhevsky, A., Sutskever, I. & Hinton, G. E. ImageNet classification with deep convolutional neural networks. *Commun. ACM* **60**(6), 84–90. https://doi.org/10.1145/3065386 (2017).
62. Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556. https://doi.org/10.48550/arXiv.1409.1556
63. He, K., et al. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770–778. IEEE. https://doi.org/10.1109/CVPR.2016.90.
64. Abbas, A. et al. The power of quantum neural networks. *Nat. Comput. Sci.* **1**, 403–409. https://doi.org/10.1038/s43588-021-00084-1 (2021).
65. Long, C. et al. Hybrid quantum-classical-quantum convolutional neural networks. *Sci. Rep.* **15**, 31780. https://doi.org/10.1038/s41598-025-13417-1 (2025).
66. Aaronson, S. (2015). Quantum Machine Learning Algorithms : Read the Fine Print. URL: https://www.scottaaronson.com/papers/qml.pdf
67. Schuld, M. & Killoran, N. Quantum machine learning in feature Hilbert spaces. *Phys. Rev. Lett.* https://doi.org/10.1103/physrevlett.122.040504 (2019).

Acknowledgements

This work is supported by the Wits-IBM collaborative project. We acknowledge funding from the CSIRNLC, NRF (SA), VRC(Wits), and DSI-SAQTi for supporting our research. We also acknowledge the Malawi University of Science and Technology (MUST) for supporting their employee, Mr Chisomo Daka, in his PhD studies.

Author contributions

Chisomo Daka is the main author, and he has conceptualized the proposed No-QCNN model. Somnath Bhattacharyya is the corresponding author, and he has contributed to manuscript compilation and concept refinement. Both authors have analyzed and presented the results.

Funding

We acknowledge funding from the CSIRNLC, NRF (SA), VRC(Wits), and DSI-SAQTi for supporting our research.

Declarations

Competing interests

The authors declare no competing interests.

Ethical approval

This study does NOT in any way involve live subjects (human or animal).

Additional information

Correspondence and requests for materials should be addressed to S.B.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026